Advanced Representation Learning
(고급표현학습)

Instructor: Donghyeon Cho

The Department of Computer Science

Hanyang University

한양대학교
HANYANG UNIVERSITY

Contents of Table

- 1. Key Ingredients of Contrastive Learning
- 2. Augmentation
- 3. Learning from Parallel Augmentation
  - SimCLR
  - BYOL
  - Barlow Twins

Key Ingredients of Contrastive Learning

Heavy Data Augmentation

- Needed for creating **noise versions** of given training data
  - As **positive** samples
  - **Without modifying semantic** meanings
- Proper data augmentation setup is critical for learning good and **generalizable** embedding features

Large Batch Size

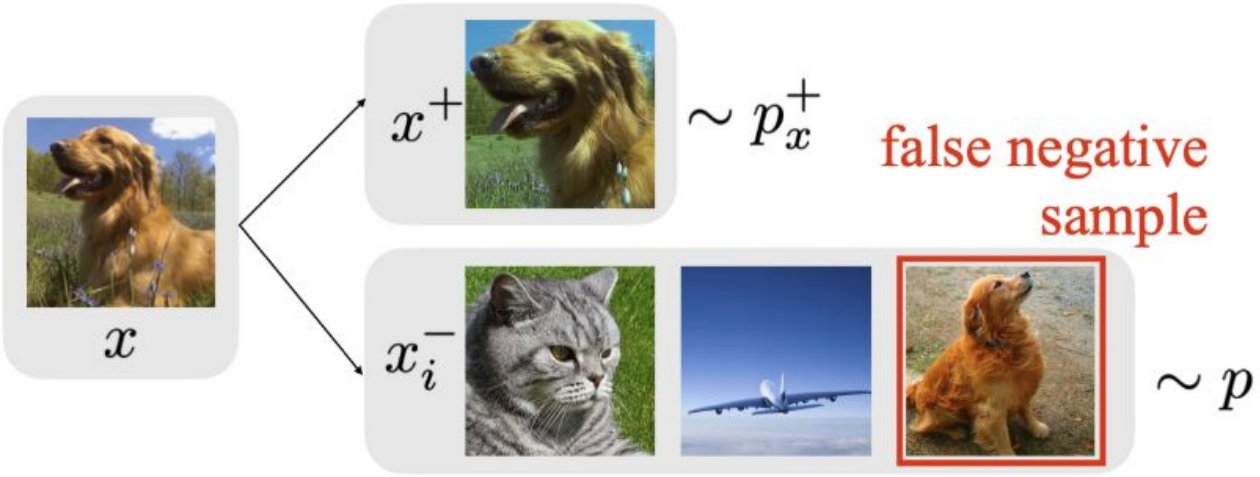
- Provides a large number of in-batch negative samples
- Increases the **diversity** of negative examples
- **Reduces false similarity** between different instances
- Encourages learning more **discriminative** representations
- Essential for methods relying on InfoNCE (e.g., SimCLR, CLIP)

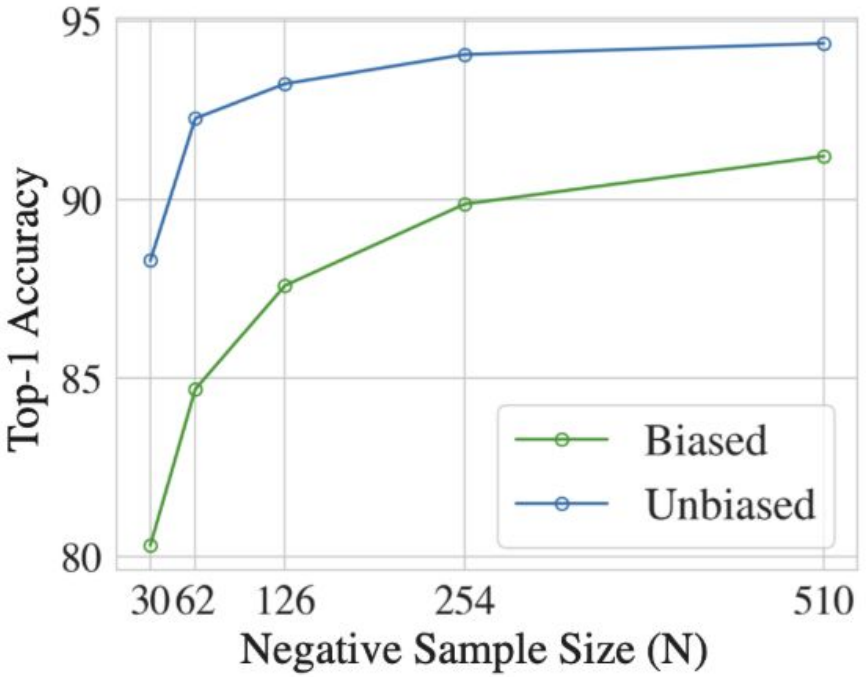
Hard Negative Mining

- Hard **negatives** are **close to the anchor** in the embedding space
- Provide more **informative** supervision than easy negatives
- Improve **discriminative** representation learning
- **Easy in supervised** learning using class labels
- **Challenging in self-supervised** learning
  - No ground-truth labels
  - Hard negatives are difficult to identify
  - Larger batches (or memory banks) increase the chance of finding hard negatives
  - Trade-off: Better negatives vs. higher memory cost

Hard Negative Mining

- Sampling bias can refers to false negative samples
  - → lead to a big performance drop





Chuang et al., 2020, Debiased Contrastive Learning

Debiased Contrastive Loss

- Problem: Random negative sampling may include **false negatives**.
- Idea: Estimate the **true negative distribution**.

Debiased Contrastive Loss

- Let us assume the probability of anchor class c is uniform

$$\rho(c) = \eta^+$$

- Probability of observing a different class

$$\eta^- = 1 - \eta^+$$

- The probability of observing a positive example for $\mathbf{x}$ is $p_x^+(\mathbf{x}') = p(\mathbf{x}'|\mathbf{h}_{x'} = \mathbf{h}_x)$;
- The probability of getting a negative sample for $\mathbf{x}$ is $p_x^-(\mathbf{x}') = p(\mathbf{x}'|\mathbf{h}_{x'} \neq \mathbf{h}_x)$.

The actual sampling data distribution

$$p(x') = p(x', h_{x'} = h_x) + p(x', h_{x'} \neq h_x) \Rightarrow p(x') = p(h_{x'} = h_x)\, p(x' \mid h_{x'} = h_x) + p(h_{x'} \neq h_x)\, p(x' \mid h_{x'} \neq h_x).$$

$$\Rightarrow p(\mathbf{x}') = \eta^+ p_x^+(\mathbf{x}') + \eta^- p_x^-(\mathbf{x}') \qquad p_x^-(\mathbf{x}') = (p(\mathbf{x}') - \eta^+ p_x^+(\mathbf{x}'))/\eta^-$$

$$\text{True Negative} = \frac{\text{Random Sampling} - \text{Positive Contribution}}{\eta^-}$$

Debiased Contrastive Loss

- Let us assume the probability of anchor class c is uniform

$$\rho(c) = \eta^+$$

- Probability of observing a different class

$$\eta^- = 1 - \eta^+$$

- The probability of observing a positive example for $\mathbf{x}$ is $p_x^+(\mathbf{x}') = p(\mathbf{x}'|\mathbf{h}_{x'} = \mathbf{h}_x)$;
- The probability of getting a negative sample for $\mathbf{x}$ is $p_x^-(\mathbf{x}') = p(\mathbf{x}'|\mathbf{h}_{x'} \neq \mathbf{h}_x)$.

The actual sampling data distribution

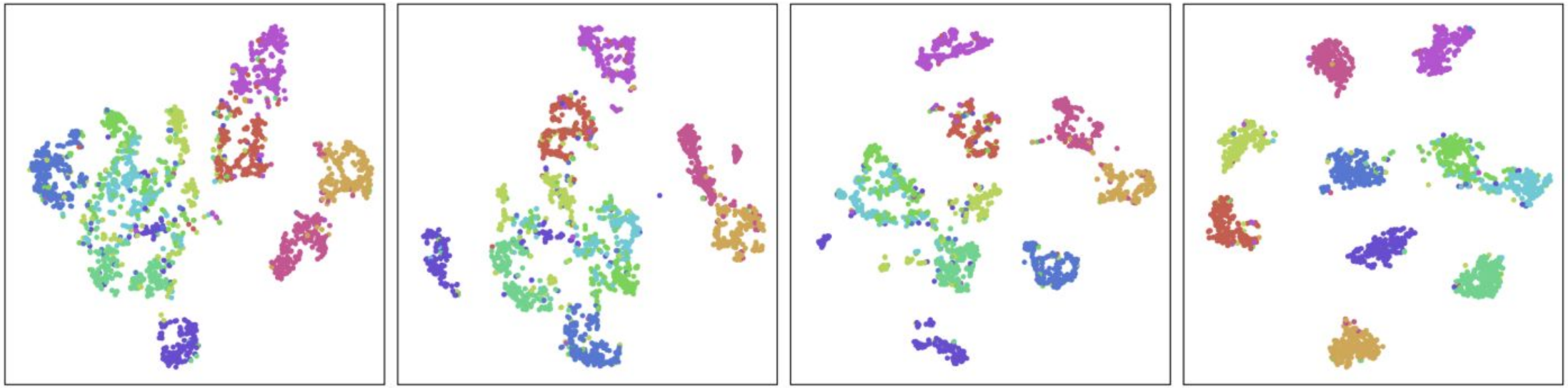
$$\text{True Negative} = \frac{\text{Random Sampling} - \text{Positive Contribution}}{\eta^-}$$

$$g(\mathbf{x}, \{\mathbf{u}_i\}_{i=1}^N, \{\mathbf{v}_i\}_{i=1}^M) = \max\left\{\frac{1}{\eta^-}\left(\frac{1}{N}\sum_{i=1}^N \exp(f(\mathbf{x})^\top f(\mathbf{u}_i)) - \frac{\eta^+}{M}\sum_{i=1}^M \exp(f(\mathbf{x})^\top f(\mathbf{v}_i))\right), \exp(-1/\tau)\right\}$$

$$\mathcal{L}_{\text{debias}}^{N,M}(f) = \mathbb{E}_{\mathbf{x},\{\mathbf{u}_i\}_{i=1}^N \sim p;\ \mathbf{x}^+,\{\mathbf{v}_i\}_{i=1}^M \sim p^+}\left[-\log \frac{\exp(f(\mathbf{x})^\top f(\mathbf{x}^+))}{\exp(f(\mathbf{x})^\top f(\mathbf{x}^+)) + Ng(x, \{\mathbf{u}_i\}_{i=1}^N, \{\mathbf{v}_i\}_{i=1}^M)}\right]$$

Debiased Contrastive Loss

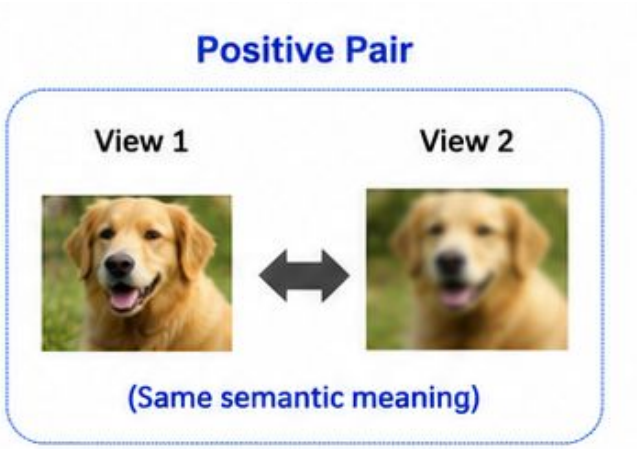
- t-SNE visualization of learned representation with debiased contrastive learning.



Augmentation

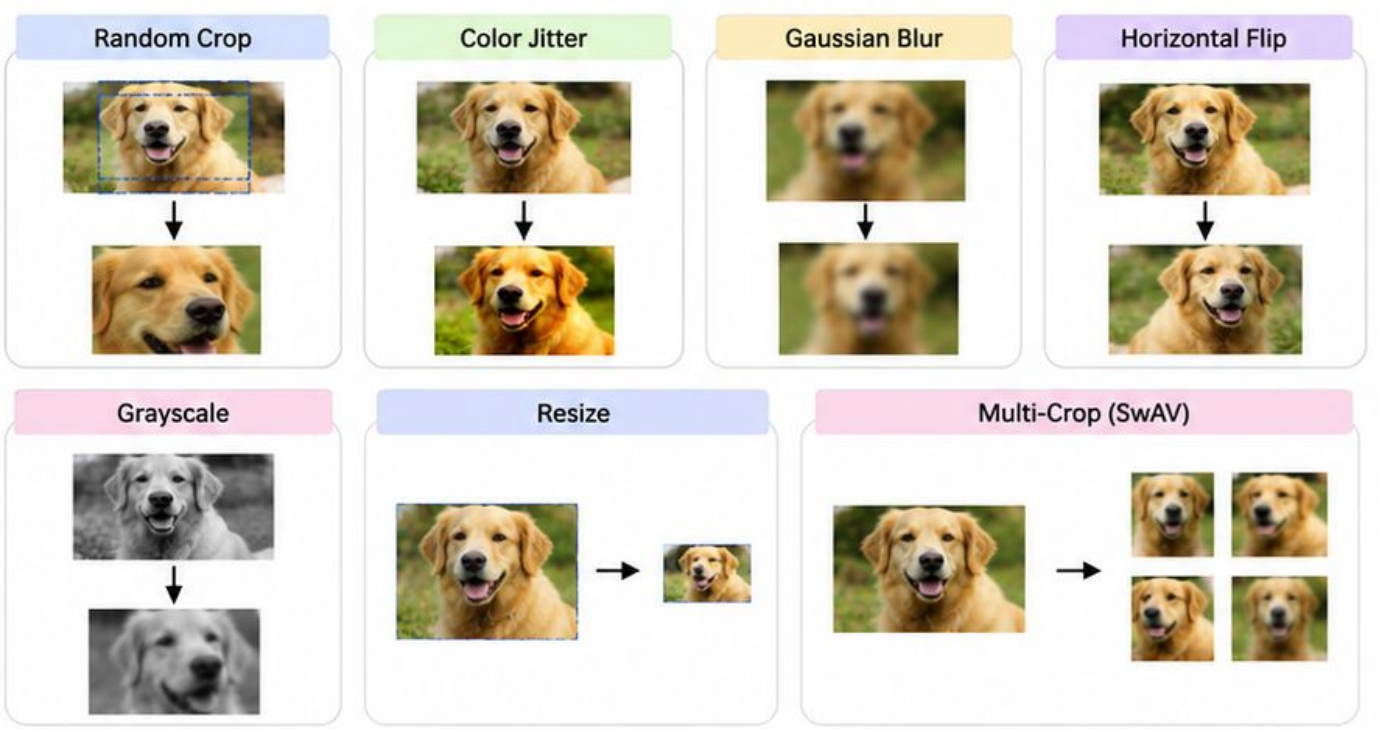
Image Augmentations

- Image augmentations create diverse views of the same image while preserving semantic meaning.
- Key Points
  - Create different **views** of the same image
  - Significantly change **visual appearance**
  - Preserve the **semantic meaning**
  - Form **positive pairs** for contrastive learning



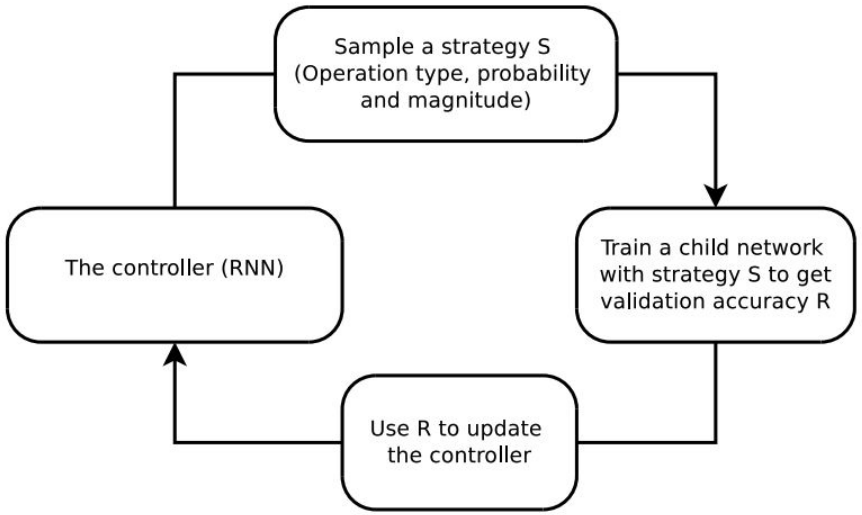
Basic Image Augmentations

- Random Resized Crop
- Color Jitter
- Gaussian Blur
- Horizontal Flip
- Grayscale
- Resize



Augmentation Strategies

- AutoAugment
  - Automatically searches for the **best augmentation policy**
  - Formulates augmentation search as a **reinforcement learning** problem
  - Maximizes validation accuracy
  - **Key Idea**: Learn which augmentations to apply.



Operations: ShearX/Y, TranslateX/Y, Rotate, AutoContrast, Invert, Equalize, Solarize, Posterize, Contrast, Color, Brightness, Sharpness, Cutout, Sample Pairing.

Cubuk et al., CVPR 2019, AutoAugment: Learning Augmentation Policies from Data

Augmentation Strategies

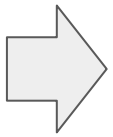
- RandAugment
  - Eliminates the expensive search process
  - Uses **randomly** selected augmentations
  - Controls all transformations with a single magnitude parameter
  - **Key Idea**: Simpler and more efficient than AutoAugment.

AutoAugment

Search

- Which operation?
- What probability?
- What magnitude?

→ Huge search space



RandAugment

Search

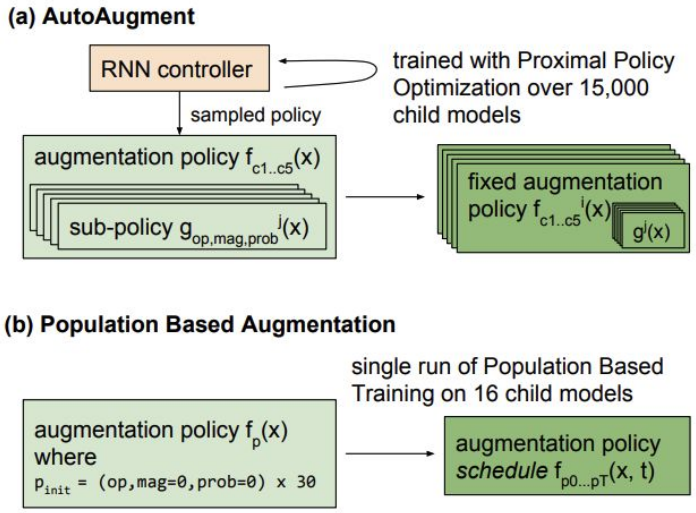
- Number of operations for an image(**N**)
- One global magnitude (**M**)

→ Very small search space

Cubuk et al., 2019, RandAugment: Practical automated data augmentation with a reduced search space

Augmentation Strategies

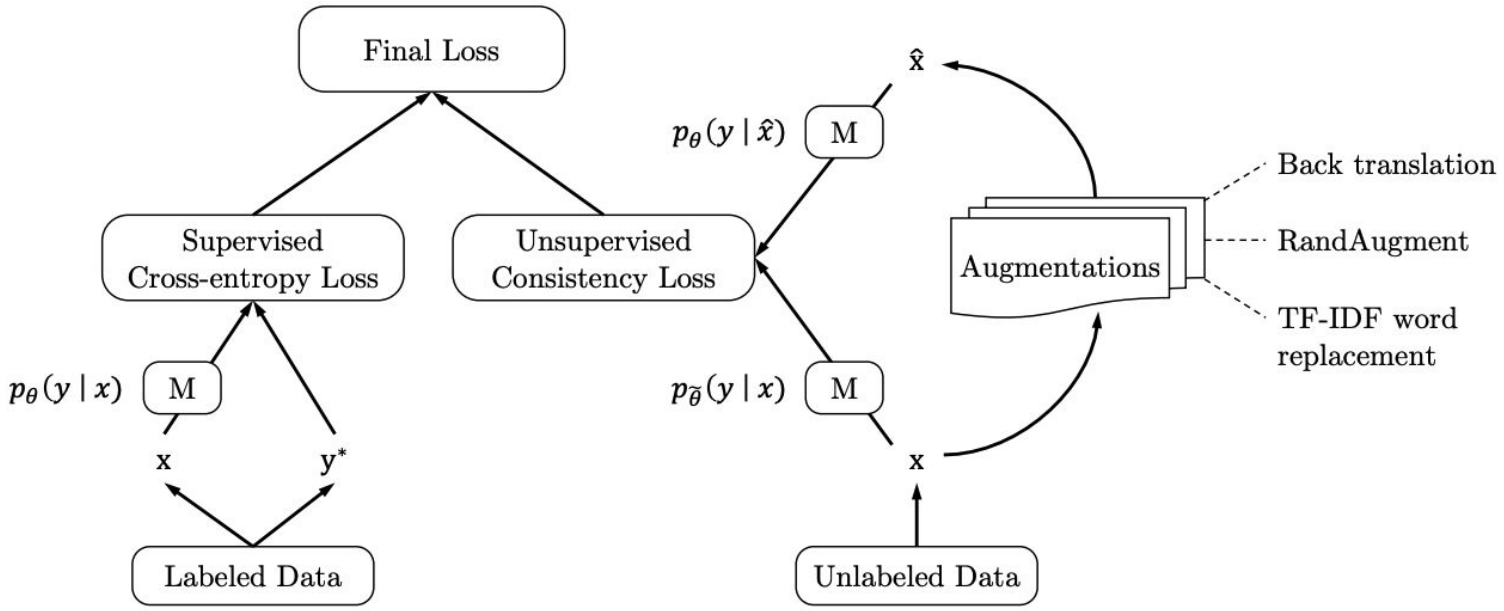
- Population Based Augmentation (PBA)
  - Evolves augmentation policies during training
  - Uses **population**-based training (PBT)
  - **Continuously updates** augmentation strategies
  - **Key Idea**: Learn augmentations **online** via evolutionary optimization.
  - In contrast to AutoAugment, PBA learns a schedule instead of a fixed policy



Ho et al., ICML 2019, Population Based Augmentation: Efficient Learning of Augmentation Policy Schedules

Augmentation Strategies

- Unsupervised Data Augmentation (UDA)
  - Applies augmentations to **unlabeled** data
  - Enforces prediction **consistency** between original and augmented images
  - Minimizes **KL** divergence between predictions
  - **Key Idea**: Learn from unlabeled data through consistency regularization.



Xie et al., NeurIPS 2020, Unsupervised Data Augmentation for Consistency Training

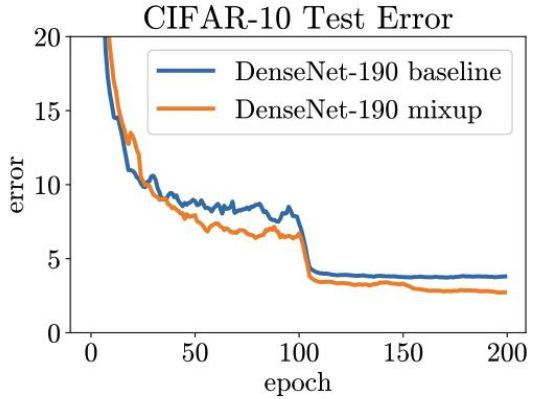
Image Mixture

- Mixup
  - It runs global-level mixture by creating a weighted pixel-wise combination of two existing images.

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j,$$
$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j,$$

Dataset	Model	ERM	*mixup*
CIFAR-10	PreAct ResNet-18	5.6	**4.2**
	WideResNet-28-10	3.8	**2.7**
	DenseNet-BC-190	3.7	**2.7**
CIFAR-100	PreAct ResNet-18	25.6	**21.1**
	WideResNet-28-10	19.4	**17.5**
	DenseNet-BC-190	19.0	**16.8**

(a) Test errors for the CIFAR experiments.



(b) Test error evolution for the best ERM and *mixup* models.

Zhang et al., ICLR 2018, mixup: Beyond Empirical Risk Minimization

Image Mixture

- Cutmix
  - Cutmix does region-level mixture by generating a new example by combining a local region of one image with the rest of the other image.

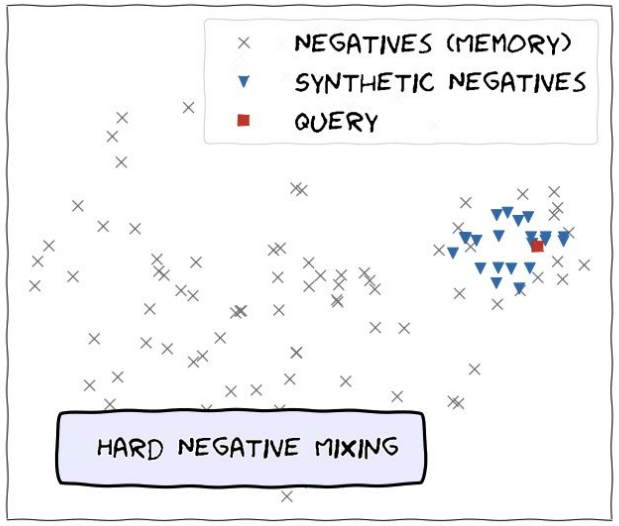
$$I_{\text{cutmix}} \leftarrow \mathbf{M}_b \odot I_1 + (1 - \mathbf{M}_b) \odot I_2, \text{ where } \mathbf{M}_b \in \{0, 1\}^I$$

	ResNet-50	Mixup [48]	Cutout [3]	CutMix
Image				
Label	Dog 1.0	Dog 0.5 Cat 0.5	Dog 1.0	Dog 0.6 Cat 0.4
ImageNet Cls (%)	76.3 (+0.0)	77.4 (+1.1)	77.1 (+0.8)	**78.6** **(+2.3)**
ImageNet Loc (%)	46.3 (+0.0)	45.8 (-0.5)	46.7 (+0.4)	**47.3** **(+1.0)**
Pascal VOC Det (mAP)	75.6 (+0.0)	73.9 (-1.7)	75.1 (-0.5)	**76.7** **(+1.1)**

Yun et al., ICCV 2019, CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features

Image Mixture

- MoCHi (Mixing of Contrastive Hard Negatives)
  - Identify the **hardest negative samples** based on similarity to the query.
  - Generate **synthetic hard negatives** by interpolating hard negative embeddings.
  - Create even harder negatives by mixing the **query** with hard negatives.
  - Improve representation learning with more informative negative samples.
  - **Key Idea**: Generate harder negatives by mixing embeddings in the feature space.

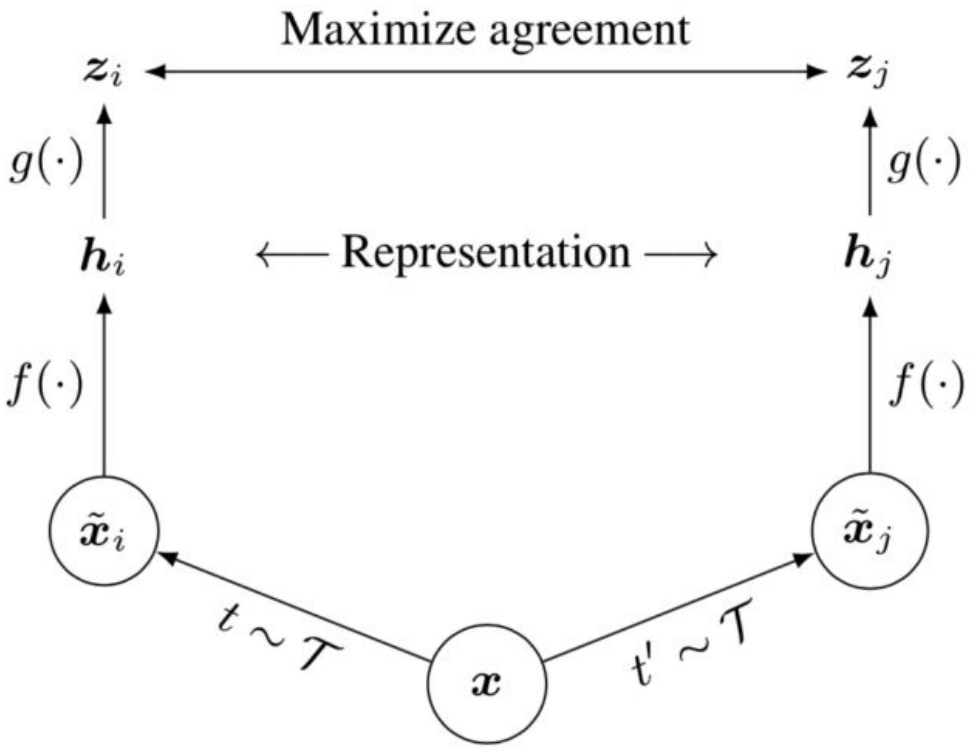


Kalantidis et al., NeurIPS 2020, Hard Negative Mixing for Contrastive Learning

Parallel Augmentation

SimCLR

- SimCLR is a simple framework for contrastive learning of visual representations.
  - maximizing agreement between differently augmented views of the same sample
  - a contrastive loss in the latent space



1. Randomly sample a minibatch of N samples and each sample is applied with two different data augmentation operations, resulting in 2N augmented samples in total.

$$\tilde{\mathbf{x}}_i = t(\mathbf{x}), \quad \tilde{\mathbf{x}}_j = t'(\mathbf{x}), \quad t, t' \sim \mathcal{T}$$

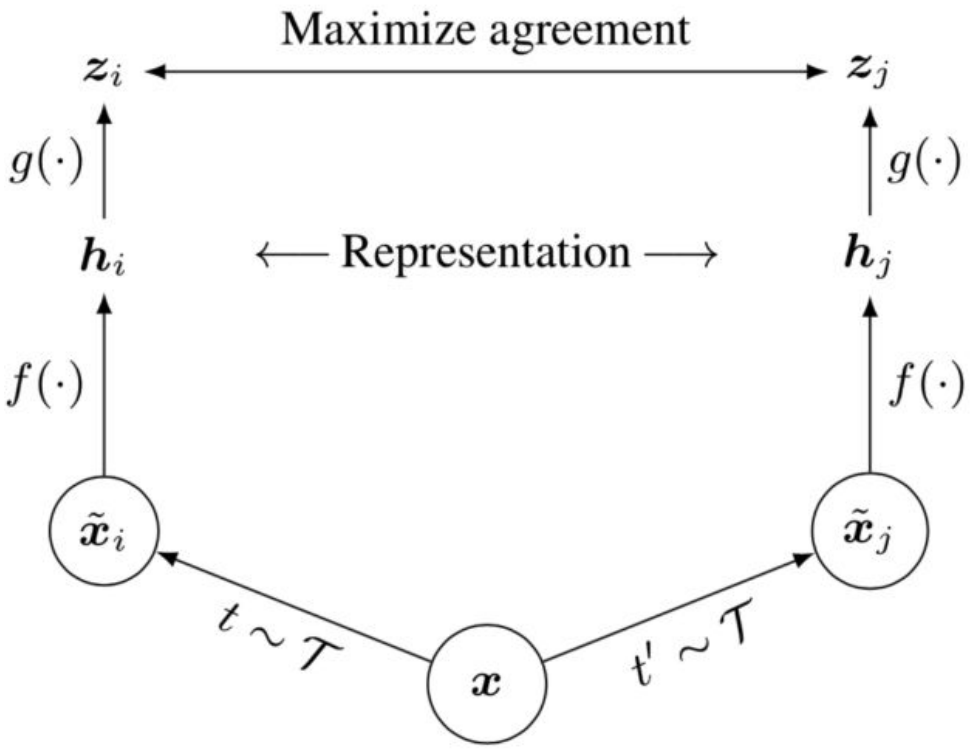
2. Given one positive pair, other 2(N-1) data points are treated as negative samples.
Representation is produced by a base encoder *f()*.

$$\mathbf{h}_i = f(\tilde{\mathbf{x}}_i), \quad \mathbf{h}_j = f(\tilde{\mathbf{x}}_j)$$

Chen et al., ICML 2020, A Simple Framework for Contrastive Learning of Visual Representations

SimCLR

- SimCLR is a simple framework for contrastive learning of visual representations.
  - maximizing agreement between differently augmented views of the same sample
  - a contrastive loss in the latent space



3. Loss operates on an extra projection layer of the representation *g()*.
   Loss is defined using cosine similarity **sim**().

$$\mathbf{z}_i = g(\mathbf{h}_i), \quad \mathbf{z}_j = g(\mathbf{h}_j)$$

$$\mathcal{L}_{\text{SimCLR}}^{(i,j)} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

SimCLR needs a **large batch size** to incorporate enough negative samples to achieve good performance.

Chen et al., ICML 2020, A Simple Framework for Contrastive Learning of Visual Representations

SimCLR

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N, constant τ, structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\boldsymbol{x}_k\}_{k=1}^{N}$ **do**
  for all $k \in \{1, \ldots, N\}$ **do**
    draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
    # the first augmentation
    $\tilde{\boldsymbol{x}}_{2k-1} = t(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k-1} = f(\tilde{\boldsymbol{x}}_{2k-1})$ # representation
    $\boldsymbol{z}_{2k-1} = g(\boldsymbol{h}_{2k-1})$ # projection
    # the second augmentation
    $\tilde{\boldsymbol{x}}_{2k} = t'(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k} = f(\tilde{\boldsymbol{x}}_{2k})$ # representation
    $\boldsymbol{z}_{2k} = g(\boldsymbol{h}_{2k})$ # projection
  end for
  for all $i \in \{1, \ldots, 2N\}$ and $j \in \{1, \ldots, 2N\}$ **do**
    $s_{i,j} = \boldsymbol{z}_i^\top \boldsymbol{z}_j / (\|\boldsymbol{z}_i\| \|\boldsymbol{z}_j\|)$ # pairwise similarity
  end for
  define $\ell(i, j)$ **as** $\ell(i,j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
  $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
  update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

Generate a **positive pair** by sampling data augmentation functions

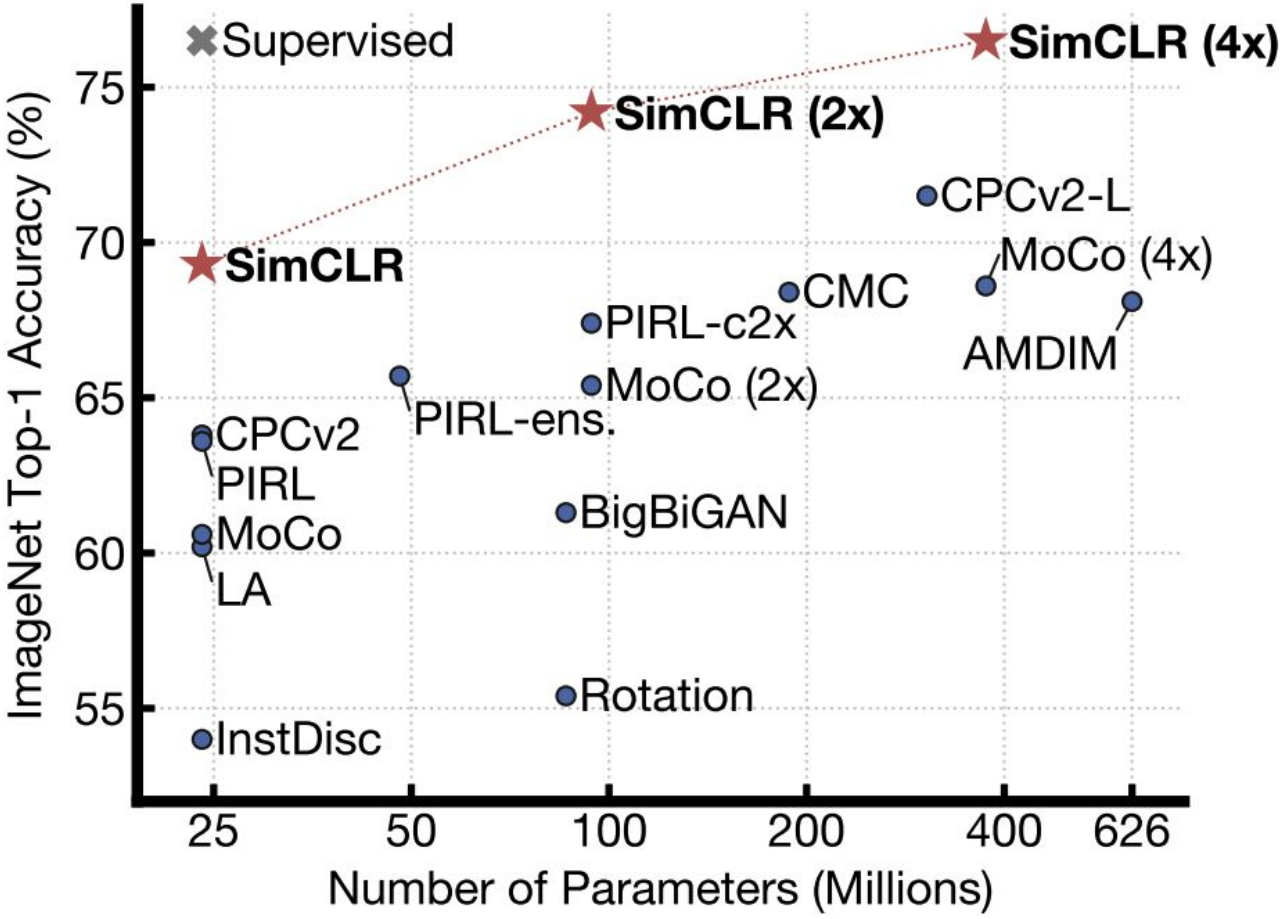
InfoNCE loss:
Use all non-positive samples in the batch as x -

Iterate through and use each of the 2N sample as reference, compute average loss

Chen et al., ICML 2020, A Simple Framework for Contrastive Learning of Visual Representations

Training Linear Classifier on SimCLR Features

- Train feature encoder on ImageNet (entire training set) using SimCLR.
- Freeze feature encoder, train a linear classifier on top with labeled data.



Chen et al., ICML 2020, A Simple Framework for Contrastive Learning of Visual Representations

Training Linear Classifier on SimCLR Features

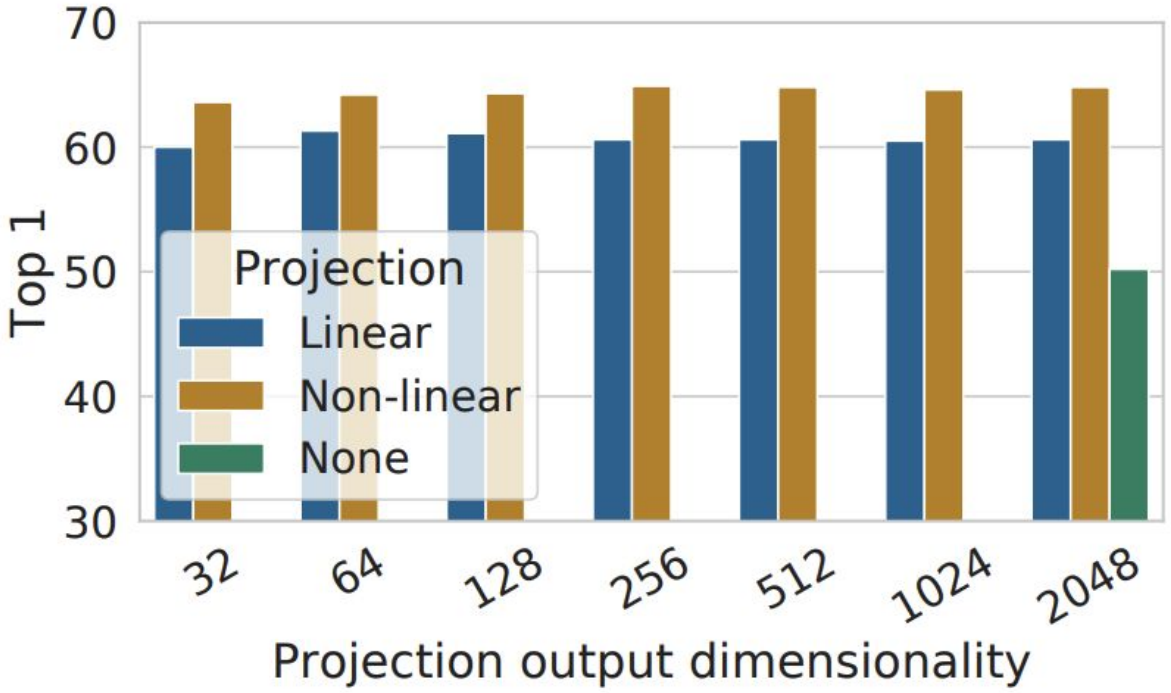
- Finetune the encoder with 1% / 10% of labeled data on ImageNet.

Method	Architecture	Label fraction (Top 5) 1%	10%
Supervised baseline	ResNet-50	48.4	80.4
Methods using other label-propagation:			
Pseudo-label	ResNet-50	51.6	82.4
VAT+Entropy Min.	ResNet-50	47.0	83.4
UDA (w. RandAug)	ResNet-50	-	88.5
FixMatch (w. RandAug)	ResNet-50	-	89.1
S4L (Rot+VAT+En. M.)	ResNet-50 (4×)	-	91.2
Methods using representation learning only:			
InstDisc	ResNet-50	39.2	77.4
BigBiGAN	RevNet-50 (4×)	55.2	78.8
PIRL	ResNet-50	57.2	83.8
CPC v2	ResNet-161(*)	77.9	91.2
SimCLR (ours)	ResNet-50	75.5	87.8
SimCLR (ours)	ResNet-50 (2×)	83.0	91.2
SimCLR (ours)	ResNet-50 (4×)	**85.8**	**92.6**

Chen et al., ICML 2020, A Simple Framework for Contrastive Learning of Visual Representations

SimCLR Design Choices: Projection Head

- Linear / non-linear projection heads improve representation learning.



A possible explanation:

- contrastive learning objective may discard useful information for downstream tasks
- representation space z is trained to be invariant to data transformation.
- by leveraging the projection head g(·), more information can be preserved in the h representation space

Chen et al., ICML 2020, A Simple Framework for Contrastive Learning of Visual Representations

SimCLR Design Choices: Large Batch Size

- Large training batch size is crucial for SimCLR!
  - Large batch size causes large memory footprint during backpropagation: requires distributed training on TPUs (ImageNet experiments)

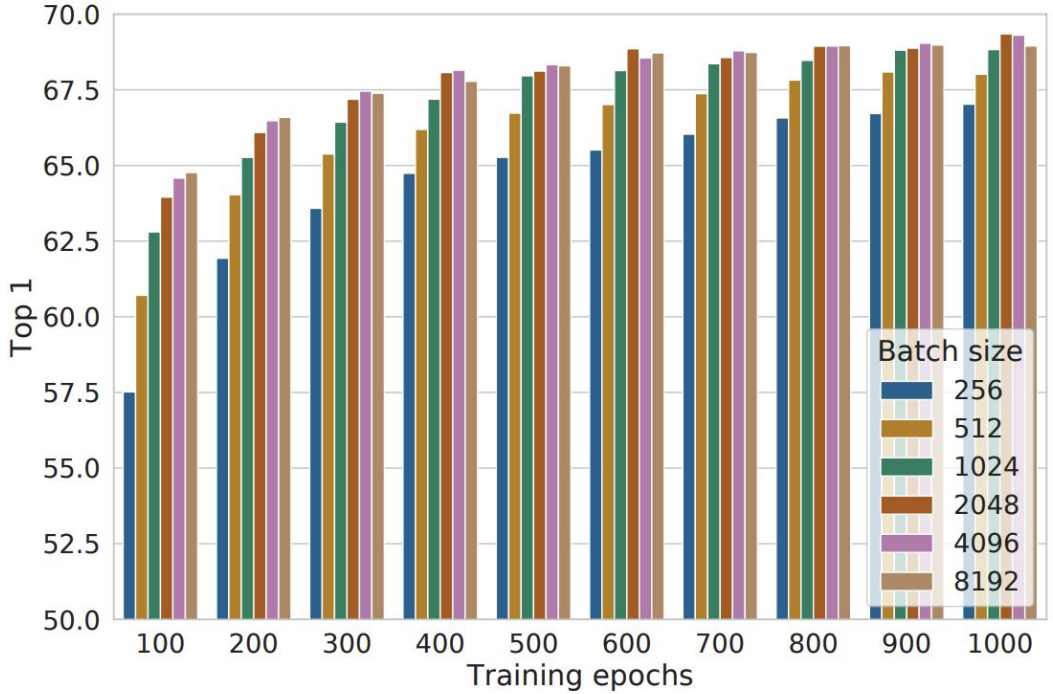
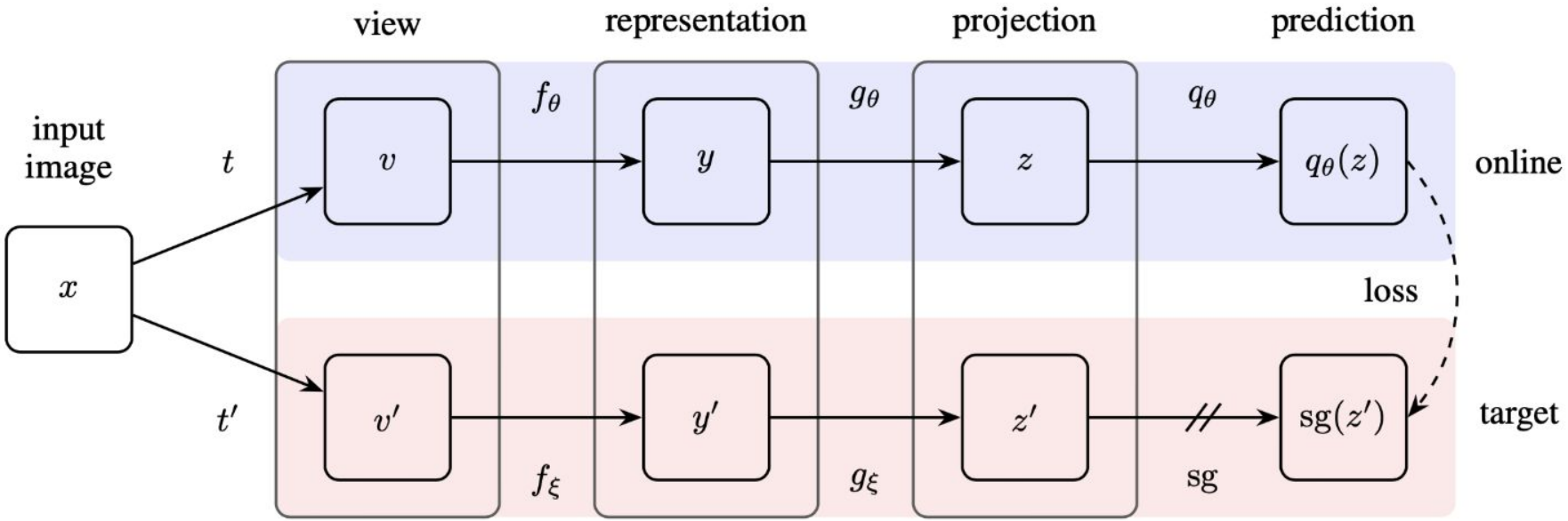


Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.[10]

Chen et al., ICML 2020, A Simple Framework for Contrastive Learning of Visual Representations

BYOL (Bootstrap Your Own Latent)

- A new state-of-the-art results without using negative samples.
- Two neural networks interact and learn from each other.
  - Online (parameterized by θ)
  - Target (parameterized by ξ)



- An encoder f_θ;
- A projector g_θ;
- A predictor q_θ.

$$\theta: \xi \leftarrow \tau\xi + (1-\tau)\theta.$$

Exponential Moving Average (EMA)

Grill et al., NeurIPS 2020, Bootstrap Your Own Latent

BYOL (Bootstrap Your Own Latent)

- Why Two Networks?
- Online Network
  - Learns through gradient descent
  - Predicts the target representation
- Target Network
  - Provides a stable learning target
  - Updated by EMA
  - No backpropagation

Grill et al., NeurIPS 2020, Bootstrap Your Own Latent

BYOL (Bootstrap Your Own Latent)

- Create two augmented views: $\mathbf{v} = t(\mathbf{x}); \mathbf{v}' = t'(\mathbf{x})$ with augmentations sampled $t \sim \mathcal{T}, t' \sim \mathcal{T}'$;
- Then they are encoded into representations, $\mathbf{y}_\theta = f_\theta(\mathbf{v}), \mathbf{y}' = f_\xi(\mathbf{v}')$;
- Then they are projected into latent variables, $\mathbf{z}_\theta = g_\theta(\mathbf{y}_\theta), \mathbf{z}' = g_\xi(\mathbf{y}')$;
- The online network outputs a prediction $q_\theta(\mathbf{z}_\theta)$;
- Both $q_\theta(\mathbf{z}_\theta)$ and $\mathbf{z}'$ are L2-normalized, giving us $\bar{q}_\theta(\mathbf{z}_\theta) = q_\theta(\mathbf{z}_\theta)/|q_\theta(\mathbf{z}_\theta)|$ and $\bar{\mathbf{z}}' = \mathbf{z}'/|\mathbf{z}'|$;
- The loss $\mathcal{L}_\theta^{\text{BYOL}}$ is MSE between L2-normalized prediction $\bar{q}_\theta(\mathbf{z})$ and $\bar{\mathbf{z}}'$;
- The other symmetric loss $\tilde{\mathcal{L}}_\theta^{\text{BYOL}}$ can be generated by switching $\mathbf{v}'$ and $\mathbf{v}$; that is, feeding $\mathbf{v}'$ to online network and $\mathbf{v}$ to target network.
- The final loss is $\mathcal{L}_\theta^{\text{BYOL}} + \tilde{\mathcal{L}}_\theta^{\text{BYOL}}$ and only parameters θ are optimized.

BYOL does not use negative pairs.
BYOL directly boostrapps the latent representation.

Grill et al., NeurIPS 2020, Bootstrap Your Own Latent

BYOL (Bootstrap Your Own Latent)

Method	Top-1	Top-5
Local Agg.	60.2	-
PIRL [35]	63.6	-
CPC v2 [32]	63.8	85.3
CMC [11]	66.2	87.0
SimCLR [8]	69.3	89.0
MoCo v2 [37]	71.1	-
InfoMin Aug. [12]	73.0	91.1
BYOL (ours)	**74.3**	**91.6**

Linear evaluation on ImageNet

Method	Top-1		Top-5	
	1%	10%	1%	10%
Supervised [77]	25.4	56.4	48.4	80.4
InstDisc	-	-	39.2	77.4
PIRL [35]	-	-	57.2	83.8
SimCLR [8]	48.3	65.6	75.5	87.8
BYOL (ours)	**53.2**	**68.8**	**78.4**	**89.0**

Semi-supervised training on ImageNet

Grill et al., NeurIPS 2020, Bootstrap Your Own Latent

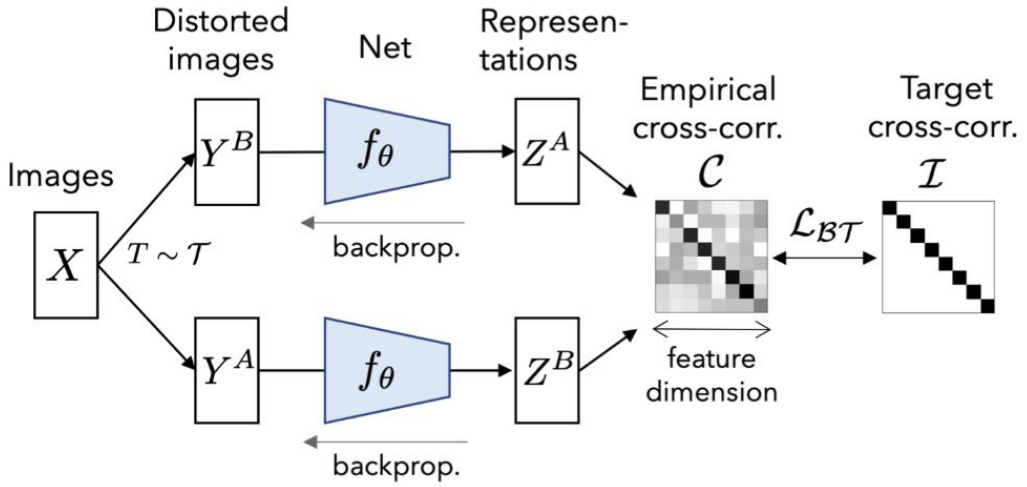
BYOL (Bootstrap Your Own Latent)

- Learn by predicting another augmented view.
  - Learn by prediction
  - No negative samples
  - Online → Target Network

Grill et al., NeurIPS 2020, Bootstrap Your Own Latent

Barlow Twins

- Generate **two augmented views** of the same image.
- **Encode** both views using a shared encoder.
- Compute the **cross-correlation** matrix between the two representations.
- Learn an **identity correlation** matrix.
- **Key Idea**: Learn invariant yet non-redundant representations without negative samples.



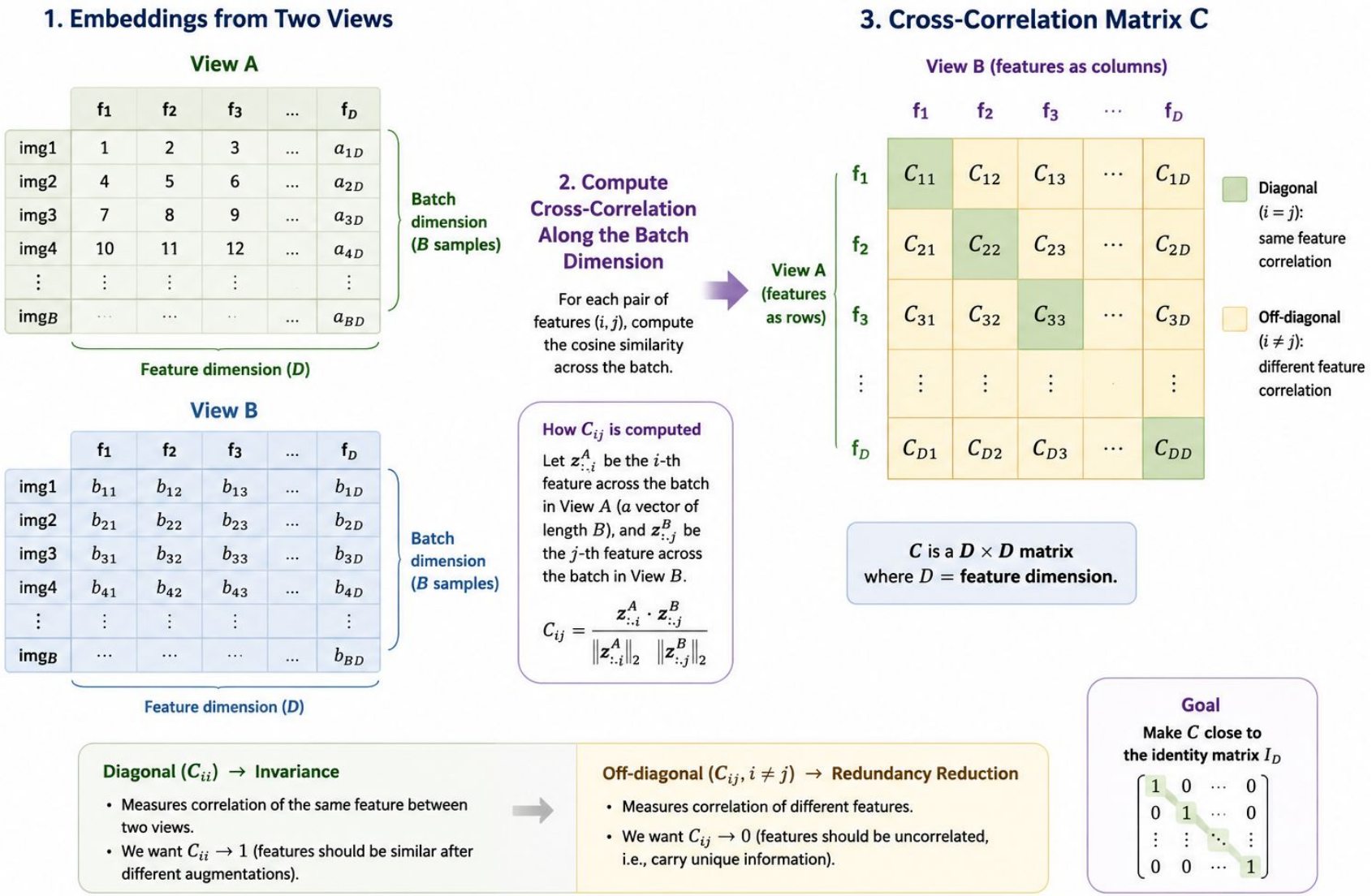
$$\mathcal{L}_{\mathrm{BT}} = \underbrace{\sum_i (1 - \mathcal{C}_{ii})^2}_{\text{invariance term}} + \lambda \underbrace{\sum_i \sum_{i \neq j} \mathcal{C}_{ij}^2}_{\text{redundancy reduction term}}$$

$$\text{where } \mathcal{C}_{ij} = \frac{\sum_b \mathbf{z}^A_{b,i} \mathbf{z}^B_{b,j}}{\sqrt{\sum_b (\mathbf{z}^A_{b,i})^2} \sqrt{\sum_b (\mathbf{z}^B_{b,j})^2}}$$

C be be a cross-correlation matrix computed between outputs from two identical networks along the batch dimension.

Zbontar et al., ICML 2021, Barlow Twins: Self-Supervised Learning via Redundancy Reduction

Barlow Twins



Zbontar et al., ICML 2021, Barlow Twins: Self-Supervised Learning via Redundancy Reduction

Barlow Twins

- Barlow Twins is competitive with SOTA methods for self-supervised learning. It naturally avoids trivial constants (i.e. collapsed representations), and is robust to different training batch sizes.

Table 1. **Top-1 and top-5 accuracies (in %) under linear evaluation on ImageNet**. All models use a ResNet-50 encoder. Top-3 best self-supervised methods are underlined.

Method	Top-1	Top-5
Supervised	76.5	
MoCo	60.6	
PIRL	63.6	-
SimCLR	69.3	89.0
MoCo v2	71.1	90.1
SimSiam	71.3	-
SwAV (w/o multi-crop)	71.8	-
BYOL	74.3	91.6
SwAV	75.3	-
Barlow Twins (ours)	73.2	91.0

Table 2. **Semi-supervised learning on ImageNet** using 1% and 10% training examples. Results for the supervised method are from (Zhai et al., 2019). Best results are in **bold**.

Method	Top-1		Top-5	
	1%	10%	1%	10%
Supervised	25.4	56.4	48.4	80.4
PIRL	-	-	57.2	83.8
SimCLR	48.3	65.6	75.5	87.8
BYOL	53.2	68.8	78.4	89.0
SwAV	53.9	**70.2**	78.5	**89.9**
Barlow Twins (ours)	**55.0**	69.7	**79.2**	89.3

Zbontar et al., ICML 2021, Barlow Twins: Self-Supervised Learning via Redundancy Reduction

Barlow Twins

- Uses Cross-Correlation loss
- No negative samples
- Aligns positive pairs only
- Removes redundancy between feature dimensions
- Learns decorrelated representations

Zbontar et al., ICML 2021, Barlow Twins: Self-Supervised Learning via Redundancy Reduction

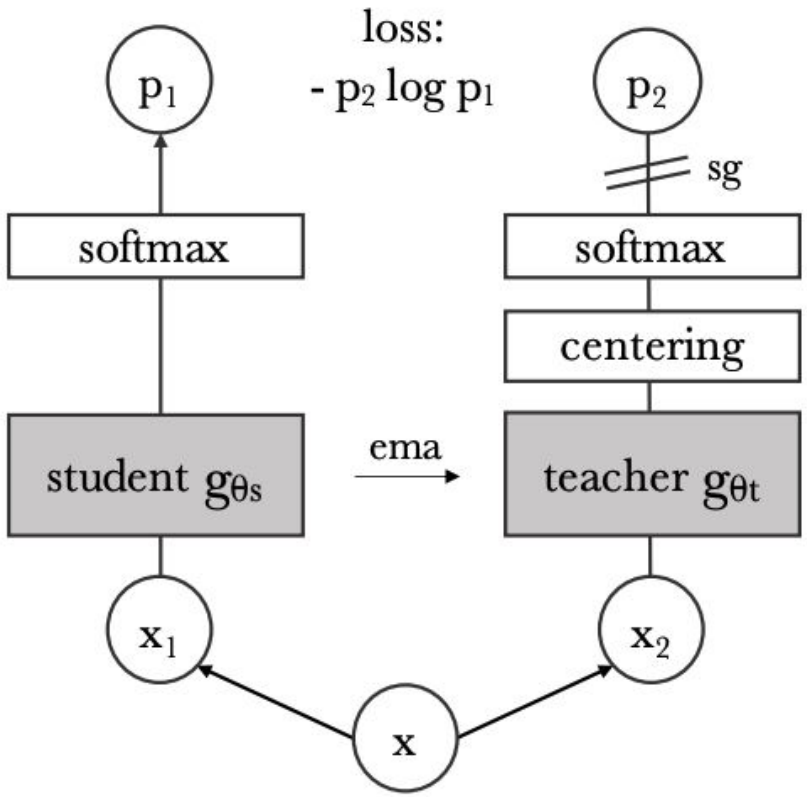
DINO: Self-Distillation with No Labels

- DINO (2021)
  - Learn visual representations through **self-distillation.**
  - Use a **student** and a **teacher** network with identical architectures.
  - Train the student to match the teacher's output distribution.
  - Update the teacher using an **EMA** of the student parameters.
- **Key Idea**: Self-distillation without labels.

Caron et al., ICCV 2021, Emerging Properties in Self-Supervised Vision Transformers

DINO: Self-Distillation with No Labels

- DINO (2021)



Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

$$c \leftarrow mc + (1-m)\frac{1}{B}\sum_{i=1}^{B} g_{\theta_t}(x_i),$$

Caron et al., ICCV 2021, Emerging Properties in Self-Supervised Vision Transformers

DINO: Self-Distillation with No Labels

- DINO (2021)

BYOL	DINO
Feature matching	Distribution matching
MSE Loss	Cross Entropy Loss
Predictor	Softmax probability
EMA Teacher	EMA Teacher
No labels	No labels

Unlike BYOL, which matches feature vectors, DINO performs self-distillation by matching the probability distributions produced by a momentum teacher network.

Caron et al., ICCV 2021, Emerging Properties in Self-Supervised Vision Transformers

DINO: Self-Distillation with No Labels

- DINO (2021)

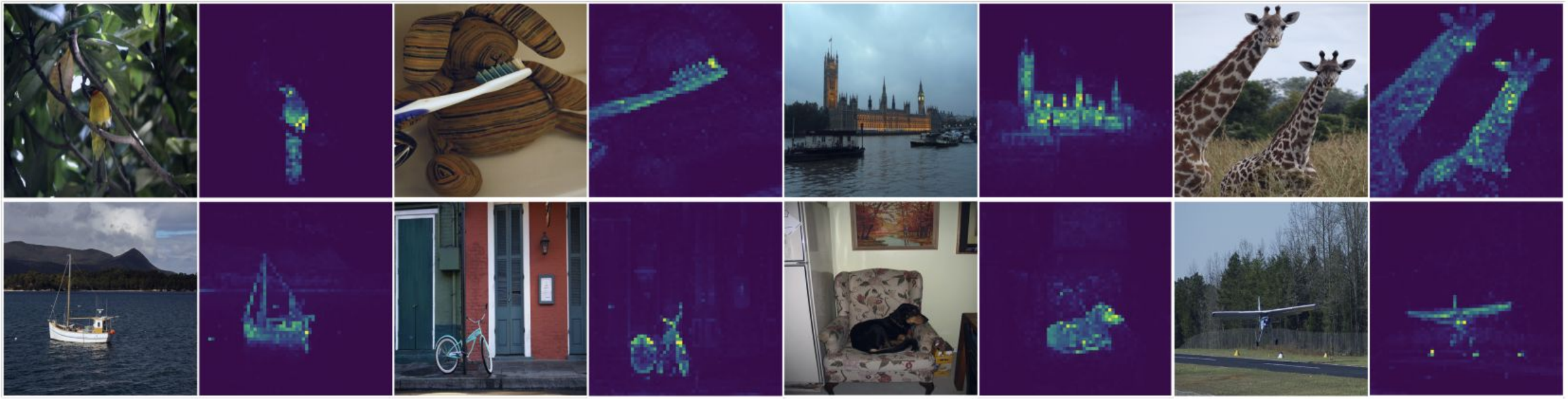


Figure 1: **Self-attention from a Vision Transformer with 8×8 patches trained with no supervision.** We look at the self-attention of the `[CLS]` token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

Caron et al., ICCV 2021, Emerging Properties in Self-Supervised Vision Transformers

DINO: Self-Distillation with No Labels

- DINO (2021)

Table 2: **Linear and k-NN classification on ImageNet.** We report top-1 accuracy for linear and k-NN evaluations on the validation set of ImageNet for different self-supervised methods. We focus on ResNet-50 and ViT-small architectures, but also report the best results obtained across architectures. * are run by us. We run the k-NN evaluation for models with official released weights. The throughput (im/s) is calculated on a NVIDIA V100 GPU with 128 samples per forward. Parameters (M) are of the feature extractor.

Method	Arch.	Param.	im/s	Linear	k-NN
Supervised	RN50	23	1237	79.3	79.3
SCLR [11]	RN50	23	1237	69.1	60.7
MoCov2 [13]	RN50	23	1237	71.1	61.9
InfoMin [54]	RN50	23	1237	73.0	65.3
BarlowT [66]	RN50	23	1237	73.2	66.0
OBoW [21]	RN50	23	1237	73.8	61.9
BYOL [23]	RN50	23	1237	74.4	64.8
DCv2 [9]	RN50	23	1237	75.2	67.1
SwAV [9]	RN50	23	1237	**75.3**	65.7
DINO	RN50	23	1237	**75.3**	**67.5**
Supervised	ViT-S	21	1007	79.8	79.8
BYOL* [23]	ViT-S	21	1007	71.4	66.6
MoCov2* [13]	ViT-S	21	1007	72.7	64.4
SwAV* [9]	ViT-S	21	1007	73.5	66.3
DINO	ViT-S	21	1007	**77.0**	**74.5**
Comparison across architectures					
SCLR [11]	RN50w4	375	117	76.8	69.3
SwAV [9]	RN50w2	93	384	77.3	67.3
BYOL [23]	RN50w2	93	384	77.4	–
DINO	ViT-B/16	85	312	78.2	76.1
SwAV [9]	RN50w5	586	76	78.5	67.1
BYOL [23]	RN50w4	375	117	78.6	–
BYOL [23]	RN200w2	250	123	79.6	73.9
DINO	ViT-S/8	21	180	79.7	**78.3**
SCLRv2 [12]	RN152w3+SK	794	46	79.8	73.1
DINO	ViT-B/8	85	63	**80.1**	77.4

Caron et al., ICCV 2021, Emerging Properties in Self-Supervised Vision Transformers

DINO: Self-Distillation with No Labels

- DINO (2021)
  - Why is DINO important?
    - Produces **semantic representations** without labels
    - Learns object-centric attention in Vision Transformers
    - Emergent segmentation and object discovery
    - Foundation of many modern SSL methods

Caron et al., ICCV 2021, Emerging Properties in Self-Supervised Vision Transformers

Can positive views be generated rather than augmented?

This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Learning Vision from Models Rivals Learning Vision from Data

Yonglong Tian[1,†] Lijie Fan[2,†,*] Kaifeng Chen[1] Dina Katabi[2] Dilip Krishnan[1] Phillip Isola[2]

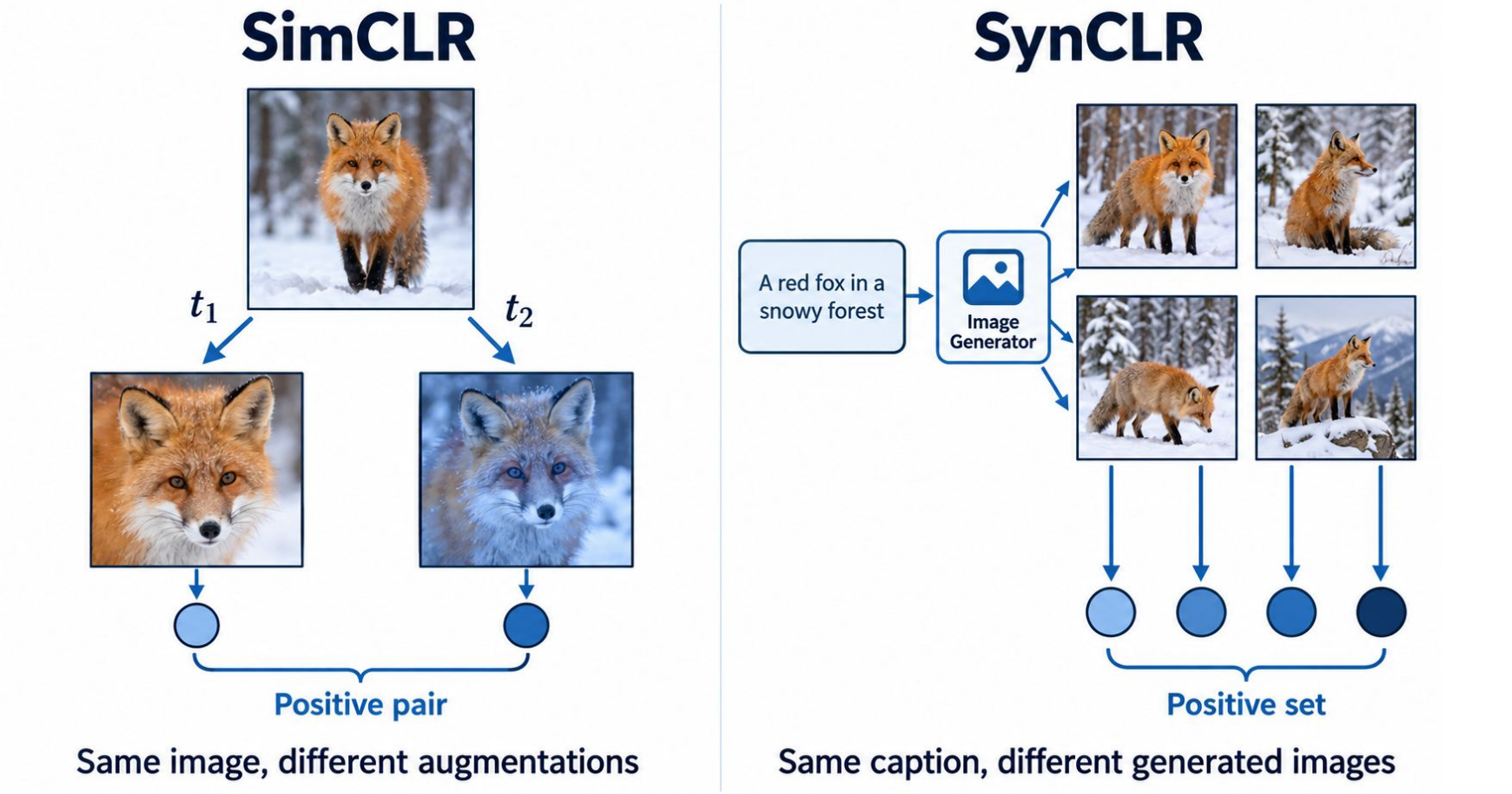
[1]Google Research, [2]MIT CSAIL, [†]equal contribution

Github Repo: https://github.com/google-research/syn-rep-learn

Tian et al., CVPR 2024, Learning Vision from Models Rivals Learning Vision from Data

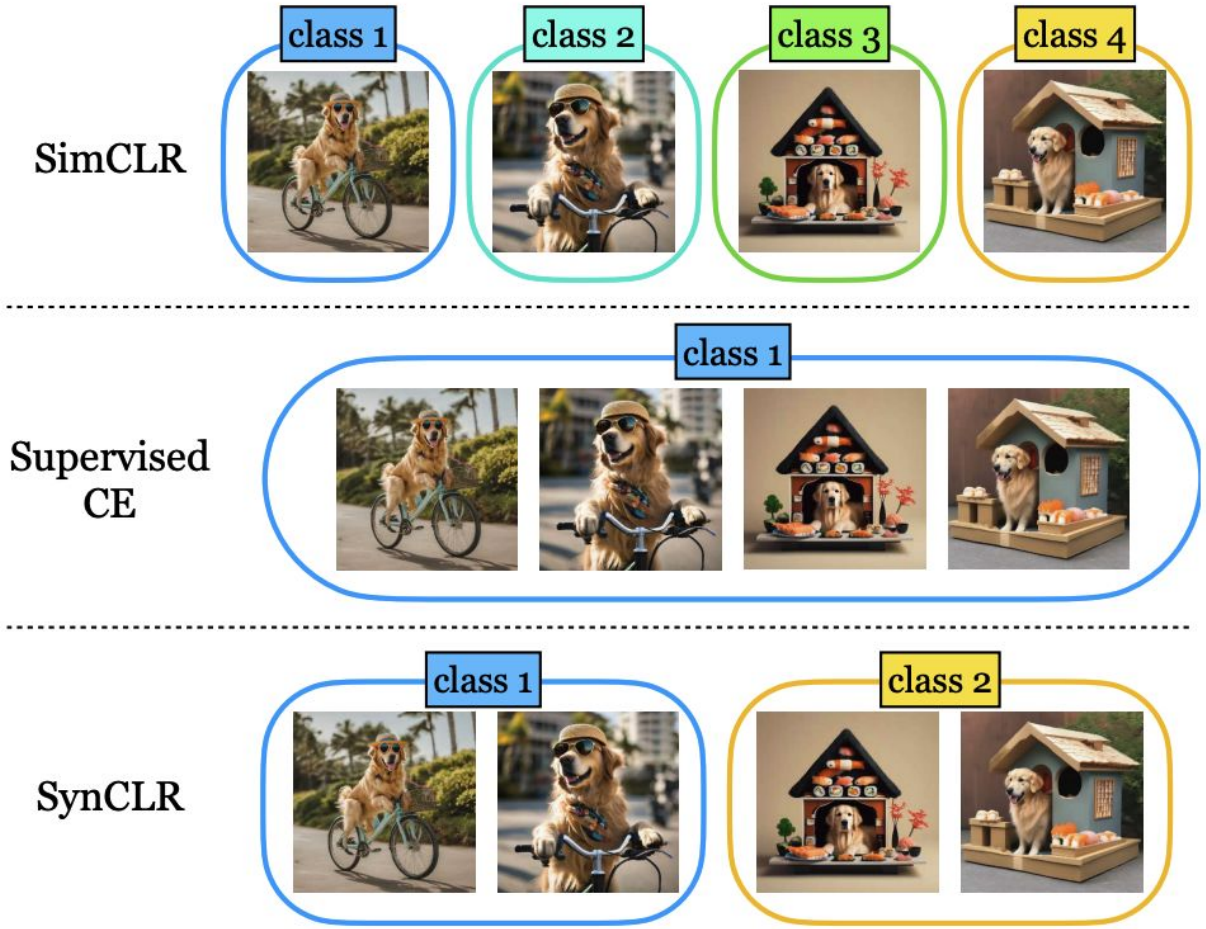
From Image Augmentation to Image Generation

- Can we generate positive views instead of augmenting an image?



- SimCLR preserves image identity.
- SynCLR preserves caption-level semantics.
- SimCLR creates positive pairs through image augmentation, whereas SynCLR generates a positive set from shared caption semantics.

What Defines a Positive Pair?



SimCLR

- Each individual image becomes a separate class.
- Semantically similar generated images are pushed apart.

Supervised Learning

- All golden retrievers belong to one class.
- Actions and backgrounds are ignored.

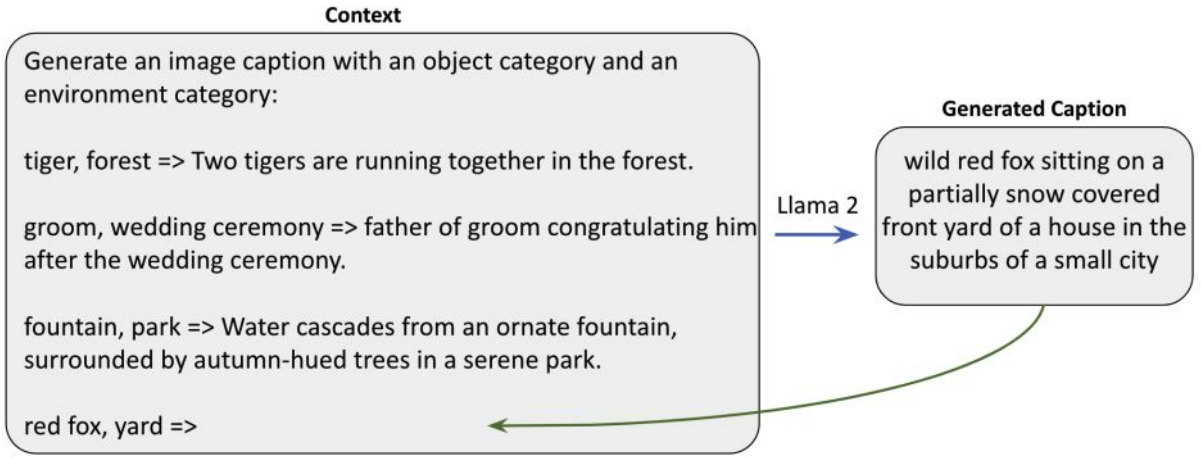
SynCLR

- Images generated from the same caption form a class.
- Captures objects, actions, and relationships.

A caption defines the granularity of a visual class.

SynCLR: Data Generation Pipeline

- Iterative Caption Generation (Generating Synthetic Captions with Llama-2)
  - Sample three examples as the initial context.
  - Llama-2 generates a new caption from the context.
  - Add the generated caption to the context.
  - Repeat the process to generate diverse captions.



Blue: Generate a new caption
Green: Add it back to the context

SynCLR: Data Generation Pipeline

- From a Caption to Positive Views
  - Use one synthetic caption as the condition.
  - Start from different random noise.
  - Generate four images for each caption.
  - Use a low CFG scale to increase diversity.
- Same caption, shared semantics
  - Different appearance and composition
  - Same objects and relationships
  - Treated as a positive set



A plate of paella, a mixed rice dish with chicken, beans, and seafood



An industrial power plant with its smokestacks belching black smoke.



A fluffy, black and white junco bird perches on a snow-covered fence, overlooking a dark forest.

Different noise produces visual diversity, while the caption preserves common semantics.

150M captions X 4 = 600 M synthetic images

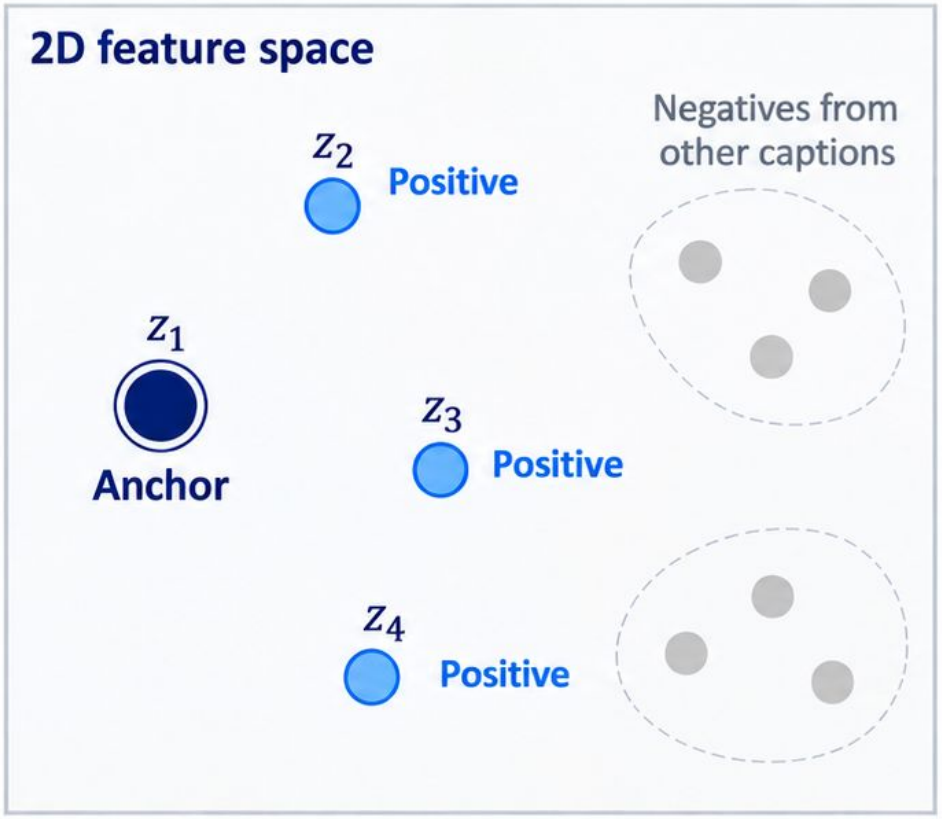
Multi-Positive Contrastive Learning

Caption c

A tiger in the forest



x_1 x_2 x_3 x_4



$$\mathbf{q}_i = \frac{\exp(\boldsymbol{a} \cdot \boldsymbol{b}_i/\tau)}{\sum_{j=1}^{K} \exp(\boldsymbol{a} \cdot \boldsymbol{b}_j/\tau)}$$

$$\mathbf{p}_i = \frac{\mathbb{1}_{\text{match}(\boldsymbol{a},\boldsymbol{b}_i)}}{\sum_{j=1}^{K} \mathbb{1}_{\text{match}(\boldsymbol{a},\boldsymbol{b}_j)}}$$

$$\mathcal{L}(\boldsymbol{a}) = H(\mathbf{p}, \mathbf{q}) = -\sum_{i=1}^{K} \mathbf{p}_i \log \mathbf{q}_i$$

For each anchor:

- Same-caption images are positives.
- Different-caption images are negatives.
- Multiple positives share caption-level semantics.

Does Learning from Models Work?

- ImageNet linear evaluation

	text	img	# imgs		ImageNet	Aircraft	Cars	DTD	Flowers	Pets	SUN397	Caltech-101	Food-101	VOC2007	Average
StableRep	real	syn	100M	ViT-B/16	75.7	59.2	83.5	80.1	97.3	88.3	74.3	94.7	85.1	87.9	83.4
CLIP	real	real	400M	ViT-B/16	80.2	59.5	86.7	79.2	98.1	93.1	78.4	94.7	92.8	89.2	85.7
				ViT-L/14	83.9	69.4	90.9	82.1	99.2	95.1	81.8	96.5	95.2	89.6	88.9
OpenCLIP	real	real	400M	ViT-B/16	78.9	61.1	92.3	81.9	98.2	91.5	77.9	95.2	90.9	88.0	86.3
			400M	ViT-L/14	82.3	67.1	94.0	83.6	98.8	92.5	81.0	96.4	93.4	88.8	88.4
			2B	ViT-L/14	83.4	71.7	95.3	85.3	99.0	94.2	82.2	97.5	94.1	88.9	89.8
DINO v2*	-	real	142M	ViT-B/14	**83.9**†	79.4	88.2	83.3	99.6	96.2	77.3	96.1	92.8	88.2	**89.0**
				ViT-L/14	**85.7**†	81.5	90.1	84.0	99.7	96.6	78.7	97.5	94.3	88.3	90.1
SynCLR	syn	syn	600M	ViT-B/16	80.7	81.7	93.8	79.9	99.1	93.6	76.2	95.3	91.6	89.4	**89.0**
				ViT-L/14	83.0	85.6	94.2	82.1	99.2	94.1	78.4	96.1	93.4	90.3	**90.4**

Table 6. **Comparison on ImageNet linear evaluation and fine-grained classificaton.** SynCLR achieves comparable results with OpenAI's CLIP and DINO v2 models, despite *only* using synthetic data. *DINO v2 modes are distilled from a ViT-g model, thus advantageous in this comparison. † we rerun only using `cls` token instead of concatenating multiple layers presented in the original DINO v2 paper [59].

Does Learning from Models Work?

- Semantic segmentation

method	pre-train data	distill	ViT-B	ViT-L
StableRep	hybrid, 100M		49.4	-
MoCo v3	real, IN1K-1M		47.3	49.1
BEiT	real, IN1K-1M+DALLE		47.1	53.3
MAE	real, IN1K-1M		48.1	53.6
iBOT	real, IN1K-1M		50.0	-
CLIP	real, WIT-400M		52.6	-
BEiT v2	real, WIT-400M, IN1K	✓	53.1	56.7
DINO v2	real, LVD-142M	✓	**54.4** †	57.5†
SynCLR	synthetic, 600M		**54.3**	**57.7** †

Table 7. **ADE20K semantic segmentation** (mIoU) using UperNet, with single scale at 512x512 resolution. † use patch size of 14x14, thus adapt to 518x518 resolution.

Synthetic pretraining also produces strong dense features.

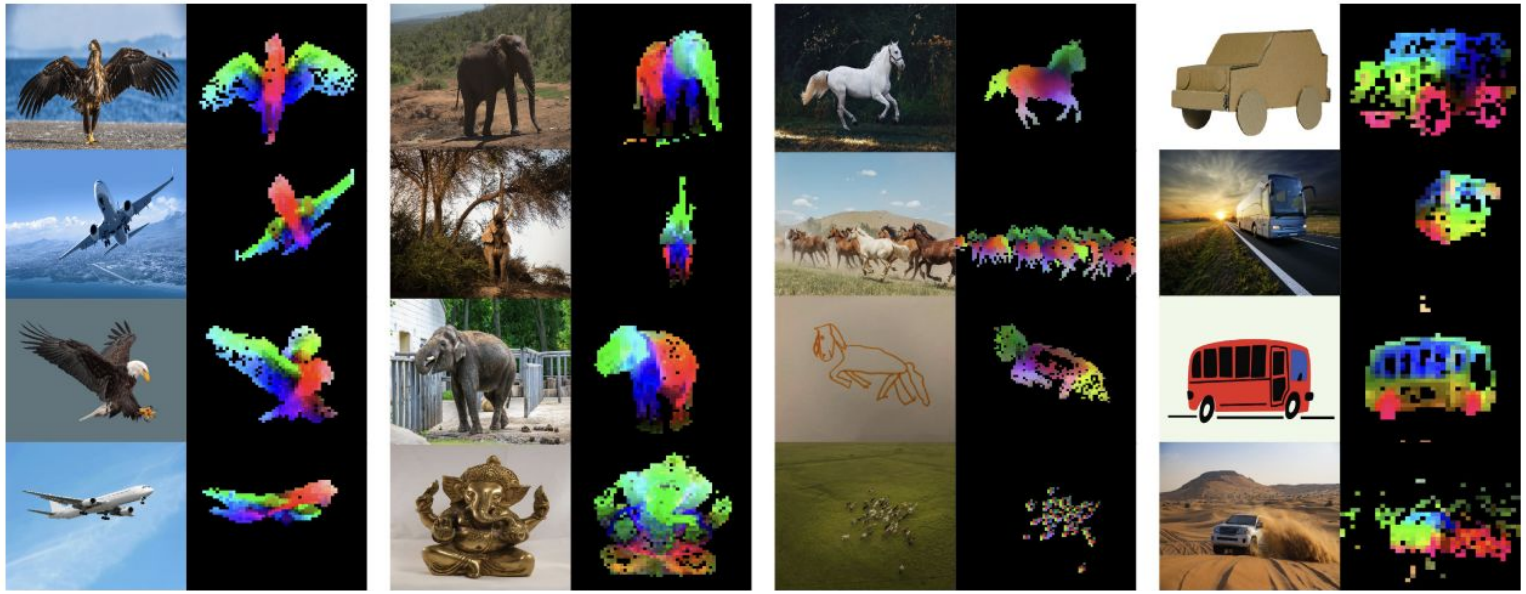
What changed?

- Hand-designed augmentation → Generative semantic views
- Positive pairs do not need to originate from the same image.
- Captions provide controllable semantic classes.
- Generative models can serve as a scalable data source.
- Representation learning can be performed without real images.

How can DINO become a general-purpose visual foundation model?

DINOv2: A Visual Foundation Model

- Learn general-purpose visual features from images alone.
  - No labels, No captions
  - Frozen features for many downstream tasks
  - Strong global and patch-level representations



PCA visualization

DINO demonstrated emergent representations.
DINOv2 aims to make them general-purpose features.

Oquab et al., TMLR 2024, DINOv2: Learning Robust Visual Features without Supervision

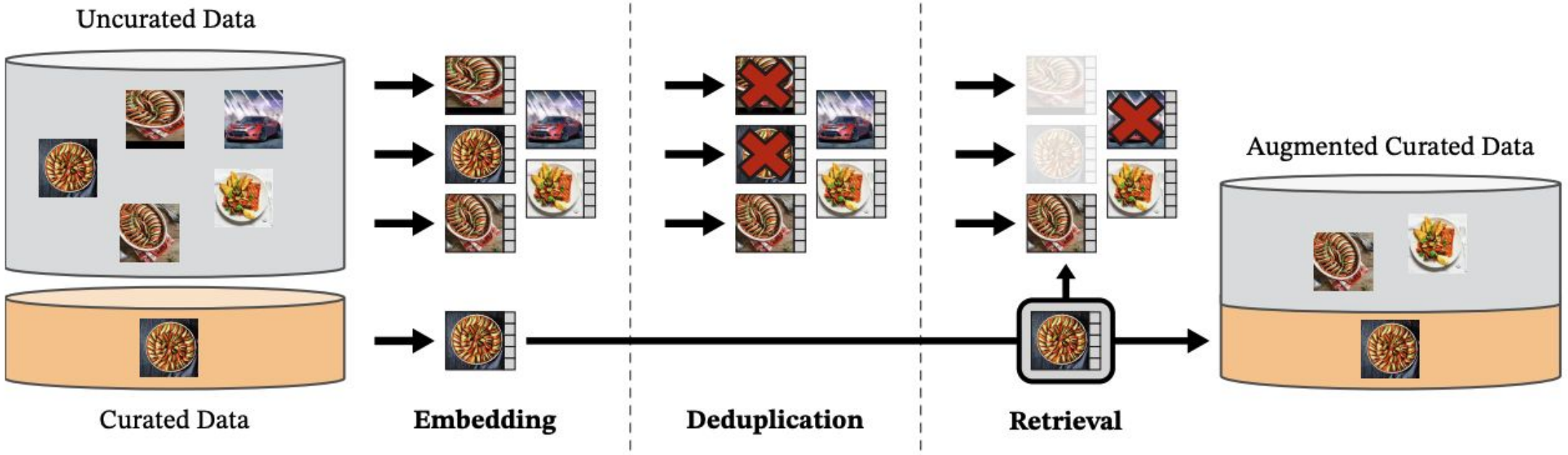
What Changed from DINO to DINOv2?

DINO	DINOv2
ImageNet-1K	Curated 142M images
Image-level self-distillation	Image-level and patch-level learning
ViT-S/B	Up to ViT-g, about 1B parameters
Semantic representation	General-purpose visual features
Research method	Foundation model family

DINOv2 = DINO + iBOT + KoLeo + Curated Data + Scaling

The core of DINOv2 is not a single, entirely new loss function; rather, it is an integrated system that combines existing self-supervised learning (SSL) components and stabilizes training to enable scaling up both data and model size.

Curating 142M Unlabeled Images



- 1. Start with curated and uncurated datasets
  a. Curated datasets provide diverse reference concepts.
     i. ImageNet-22k, ImageNet-1k train, Google Landmarks,
  b. Uncurated web images provide scale.
     i. 1.2B images from webs
- 2. Remove duplicates (Pizzi et al., CVPR 2022)
  a. Map images into an embedding space.
  b. Remove near-duplicate images.
- 3. Retrieve similar images
  a. Match uncurated images to curated examples (use self-supervised ViT-H/16).
  b. Rebalance the data around useful visual concepts.

- LVD-142M: 142 million curated images without manual labels
- Scaling requires more data, but useful scaling requires curated data.

Task	Dataset / Split	Images	Retrieval	Retrieved	Final
classification	ImageNet-22k / –	14,197,086	as is	–	14,197,086
classification	ImageNet-22k / –	14,197,086	sample	56,788,344	56,788,344
classification	ImageNet-1k / train	1,281,167	sample	40,997,344	40,997,344
fine-grained classif.	Caltech 101 / train	3,030	cluster	2,630,000	1,000,000
fine-grained classif.	CUB-200-2011 / train	5,994	cluster	1,300,000	1,000,000
fine-grained classif.	DTD / train1	1,880	cluster	1,580,000	1,000,000
fine-grained classif.	FGVC-Aircraft / train	3,334	cluster	1,170,000	1,000,000
fine-grained classif.	Flowers-102 / train	1,020	cluster	1,060,000	1,000,000
fine-grained classif.	Food-101 / train	75,750	cluster	21,670,000	1,000,000
fine-grained classif.	Oxford-IIIT Pet / trainval	3,680	cluster	2,750,000	1,000,000
fine-grained classif.	Stanford Cars / train	8,144	cluster	7,220,000	1,000,000
fine-grained classif.	SUN397 / train1	19,850	cluster	18,950,000	1,000,000
fine-grained classif.	Pascal VOC 2007 / train	2,501	cluster	1,010,000	1,000,000
segmentation	ADE20K / train	20,210	cluster	20,720,000	1,000,000
segmentation	Cityscapes / train	2,975	cluster	1,390,000	1,000,000
segmentation	Pascal VOC 2012 (seg.) / trainaug	1,464	cluster	10,140,000	1,000,000
depth estimation	Mapillary SLS / train	1,434,262	as is	–	1,434,262
depth estimation	KITTI / train (Eigen)	23,158	cluster	3,700,000	1,000,000
depth estimation	NYU Depth V2 / train	24,231	cluster	10,850,000	1,000,000
depth estimation	SUN RGB-D / train	4,829	cluster	4,870,000	1,000,000
retrieval	Google Landmarks v2 / train (clean)	1,580,470	as is	–	1,580,470
retrieval	Google Landmarks v2 / train (clean)	1,580,470	sample	6,321,880	6,321,880
retrieval	AmsterTime / new	1,231	cluster	960,000	960,000
retrieval	AmsterTime / old	1,231	cluster	830,000	830,000
retrieval	Met / train	397,121	cluster	62,860,000	1,000,000
retrieval	Revisiting Oxford / base	4,993	cluster	3,680,000	1,000,000
retrieval	Revisiting Paris / base	6,322	cluster	3,660,000	1,000,000
					142,109,386

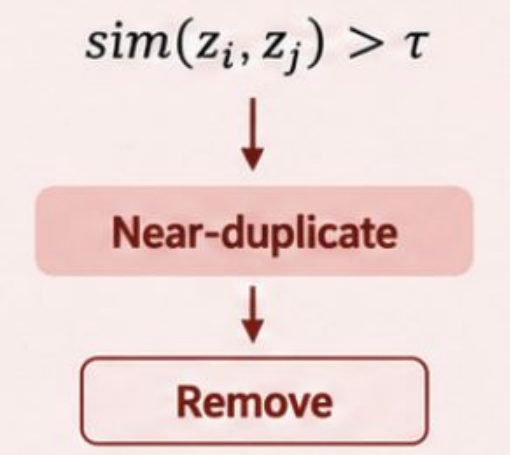
Table 15: **Composition of our LVD-142M dataset.** We report the list of datasets and associated splits used to build the dataset, how they were included (as is without retrieval or via sample-based or cluster-based retrieval). For retrievals, we indicate the actual number of retrieved images and the final number included in the dataset. We chose to include as many datasets as possible in the pretraining data in order to cover as many domains as possible. We kept a few datasets aside in order to evaluate performance outside of the pretraining domain. More details about dataset usages can be found in Table 18.

Dataset	Pretraining (as is)	Retrieving pretraining data	Eval.	Task	Citation
ImageNet-1k	✗	✓	✓	Classif.	(Russakovsky et al., 2015)
ImageNet-22k	✓	✓	✗		(Deng et al., 2009)
ImageNet-V2	✗	✗	✓	Classif.	(Recht et al., 2019)
ImageNet-ReaL	✗	✗	✓	Classif.	(Beyer et al., 2020)
ImageNet-A	✗	✗	✓	Classif.	(Hendrycks et al., 2021b)
ImageNet-C	✗	✗	✓	Classif.	(Hendrycks & Dietterich, 2019)
ImageNet-R	✗	✗	✓	Classif.	(Hendrycks et al., 2021a)
ImageNet-Sk.	✗	✗	✓	Classif.	(Wang et al., 2019)
Food-101	✗	✓	✓	Classif.	(Bossard et al., 2014)
CIFAR-10	✗	✓	✓	Classif.	(Krizhevsky et al., 2009)
CIFAR-100	✗	✓	✓	Classif.	(Krizhevsky et al., 2009)
SUN397	✗	✓	✓	Classif.	(Xiao et al., 2010)
StanfordCars	✗	✓	✓	Classif.	(Krause et al., 2013)
FGVC-Aircraft	✗	✓	✓	Classif.	(Maji et al., 2013)
VOC 2007	✗	✓	✓	Classif.	(Everingham et al., 2010)
DTD	✗	✓	✓	Classif.	(Cimpoi et al., 2014)
Oxford Pets	✗	✓	✓	Classif.	(Parkhi et al., 2012)
Caltech101	✗	✓	✓	Classif.	(Fei-Fei et al., 2004)
Flowers	✗	✓	✓	Classif.	(Nilsback & Zisserman, 2008)
CUB200	✗	✓	✓	Classif.	(Welinder et al., 2010)
iNaturalist 2018	✗	✗	✓	Classif.	(Van Horn et al., 2018)
iNaturalist 2021	✗	✗	✓	Classif.	(Van Horn et al., 2021)
Places-205	✗	✗	✓	Classif.	(Zhou et al., 2014)
UCF101	✗	✗	✓	Video	(Soomro et al., 2012)
Kinetics-400	✗	✗	✓	Video	(Kay et al., 2017)
SSv2	✗	✗	✓	Video	(Goyal et al., 2017)
GLD v2	✓	✓	✗		(Weyand et al., 2020)
R-Paris	✗	✓	✓	Retrieval	(Radenović et al., 2018a)
R-Oxford	✗	✓	✓	Retrieval	(Radenović et al., 2018a)
Met	✗	✓	✓	Retrieval	(Ypsilantis et al., 2021)
Amstertime	✗	✓	✓	Retrieval	(Yildiz et al., 2022)
ADE20k	✗	✓	✓	Seg.	(Zhou et al., 2017)
Cityscapes	✗	✓	✓	Seg.	(Cordts et al., 2016)
VOC 2012	✗	✓	✓	Seg.	(Everingham et al., 2010)
Mapillary SLS	✓	✗	✗		(Warburg et al., 2020)
NYU-Depth V2	✗	✓	✓	Depth	(Silberman et al., 2012)
KITTI	✗	✓	✓	Depth	(Geiger et al., 2013)
SUN-RGBD	✗	✓	✓	Depth	(Song et al., 2015)
DollarStreet	✗	✗	✓	Fairness	(De Vries et al., 2019)
Casual Conv.	✗	✗	✓	Fairness	(Hazirbas et al., 2021)

Table 18: **List of datasets used.**

Detecting Near-Duplicate Images

- Near-duplicate detection with SSCD
  - Encode each image into a compact descriptor.
  - Compare descriptors using cosine similarity.
  - Retrieve the nearest neighbors for each image.
  - Remove images whose similarity exceeds a predefined threshold.



Pizzi et al., CVPR 2022, A self-supervised descriptor for image copy detection

Image-Level and Patch-Level Learning

- Image-level objective (Caron et al., 2021).
  - Match the student and teacher **CLS-token** distributions.
    - Use cross-entropy so that the student follows the teacher's CLS.
  - Learn global semantic representations.

$$\mathcal{L}_{DINO} = -\sum p_t \log p_s$$

- Patch-level objective (Zhou et al., 2022a).
  - Mask some student patches.
  - Use the visible teacher patches as targets.
  - Learn local and spatial representations.
  - Separate the DINO head and the iBOT head.

$$\mathcal{L}_{iBOT} = -\sum_i p_{ti} \log p_{si}$$

i is the position of the masked patch.

Caron et al., ICCV 2019, Emerging properties in self-supervised vision transformers.
Zhou et al., ICLR 2022, ibot: Image bert pre-training with online tokenizer.

Preventing Feature Collapse and Crowding

- KoLeo regularizer (Sablayrolles et al., 2019).
  - Image-level features are distributed across the entire embedding space.
  - It pushes apart representations of different images so that they do not cluster too closely together.

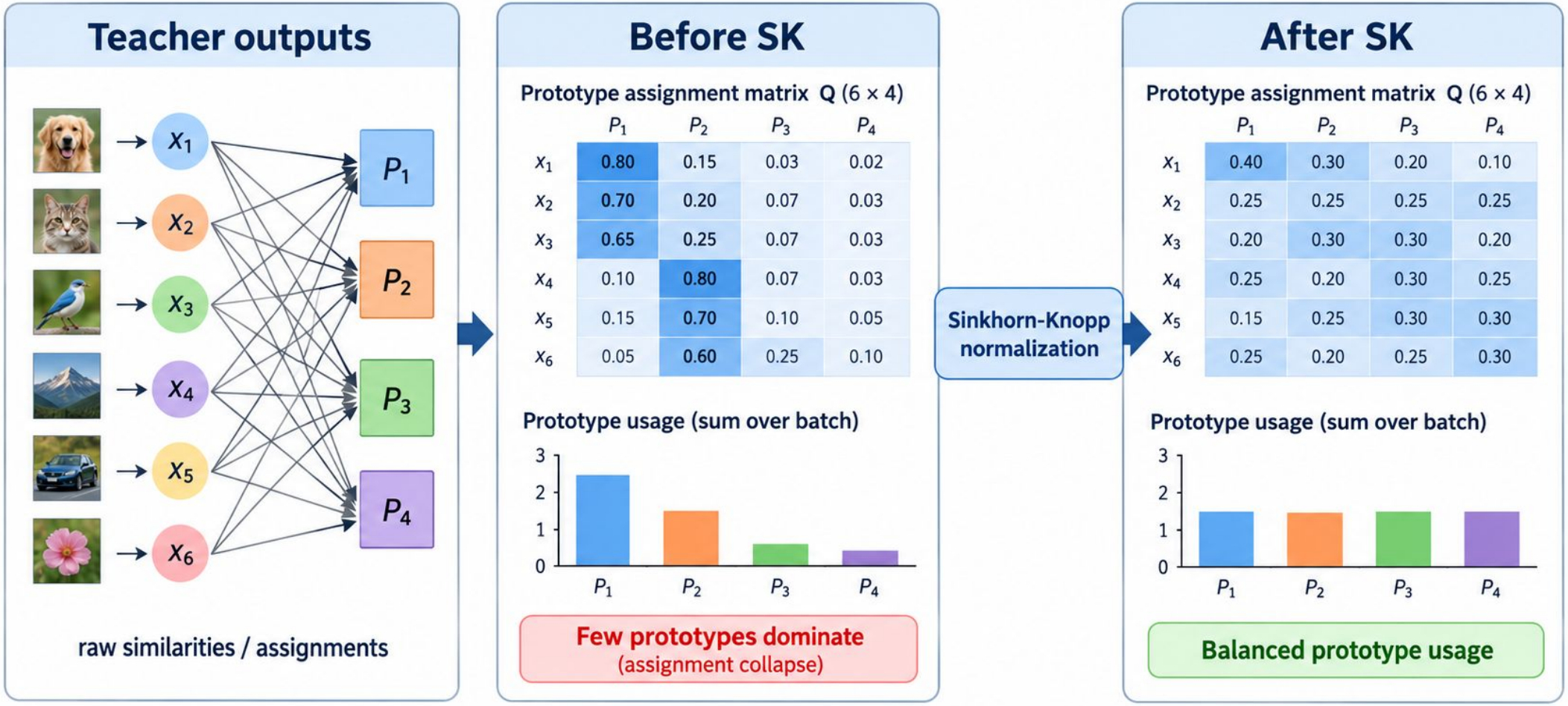
$$\mathcal{L}_{\text{koleo}} = -\frac{1}{n}\sum_{i=1}^{n}\log(d_{n,i}),$$

$$d_{n,i} = \min_{j\neq i}\|x_i - x_j\|$$

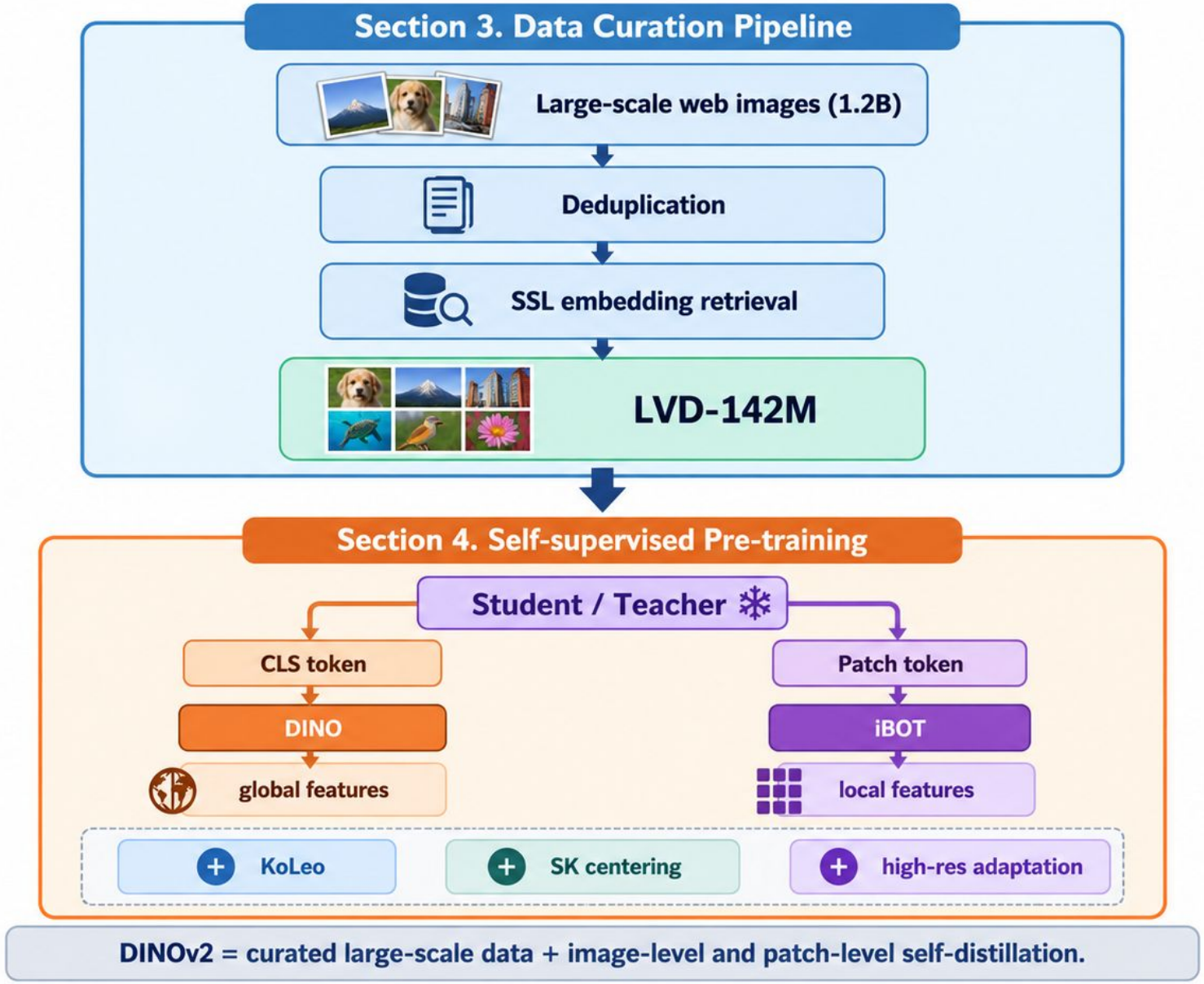
- Sinkhorn-Knopp centering (Caron et al., 2020).
  - Normalization/centering of teacher output.
  - The teacher predictions are re-normalized so that assignments are more evenly distributed across prototypes.
  - This avoids the case where many images are mapped to only a small subset of prototypes.
  - As a result, the student receives more informative and less degenerate training targets.

Sablayrolles et al., ICLR 2019, Spreading vectors for similarity search.

Sinkhorn-Knopp centering



The final weights of the projection head act as prototype-like directions.



- Only the last short training epoch is trained at 518×518

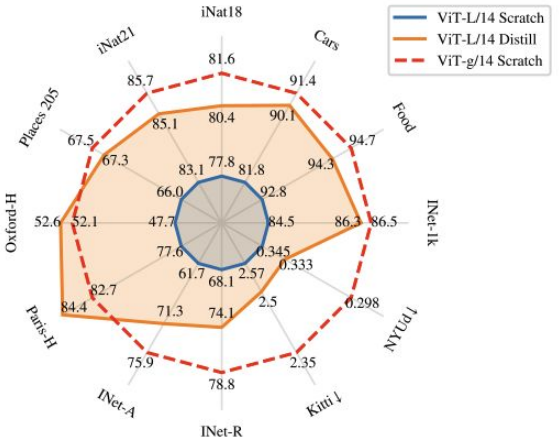
Scaling the Teacher, Distilling the Students

- Step 1
  - Train a large ViT-g teacher on LVD-142M.
- Step 2
  - Freeze the large teacher.
  - Distill its representation into smaller ViT models.

Arch	Method	INet-1k	Segm.	Depth↓	Classif.
ViT-g/14	Scratch	86.5	73.4	1.00	92.1
ViT-L/14	Scratch	84.5	72.2	1.10	90.2
ViT-L/14	Distill	**86.3**	**73.3**	**1.08**	**91.2**

Arch	Method	Finegr.	Retriev.	ARSketch	Video
ViT-g/14	Scratch	78.3	75.2	77.0	69.3
ViT-L/14	Scratch	75.8	71.3	69.5	67.3
ViT-L/14	Distill	**77.6**	**76.3**	**74.5**	**67.5**

(b) Averaged metrics on 8 vision tasks



(a) Comparison on individual metrics

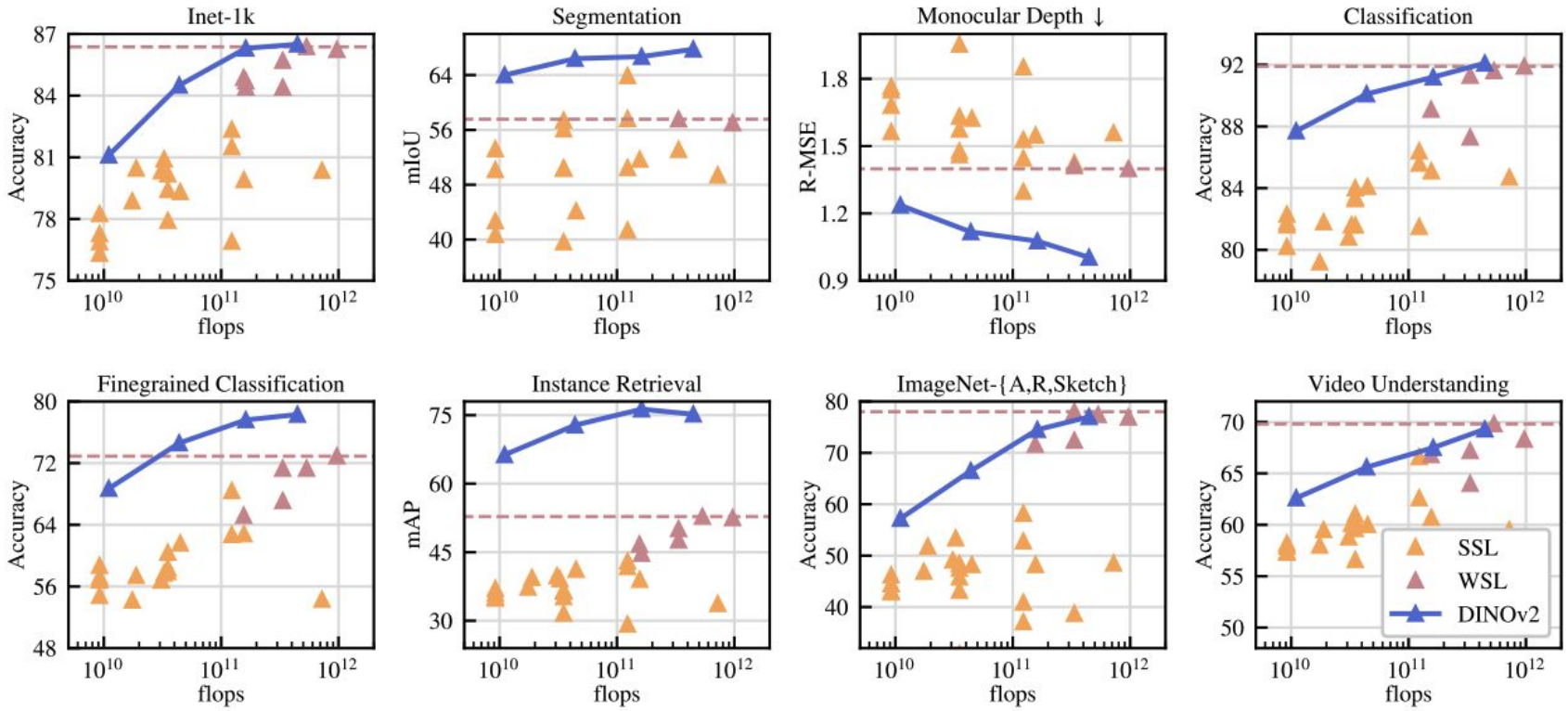
Observation

- Distilled ViT-L outperforms the same ViT-L trained from scratch.
- On some tasks, the student approaches or exceeds the teacher.

Train one large model, then transfer its representation to practical models.

One Frozen Encoder, Many Tasks

- Global features: Classification, Fine-grained recognition, Image retrieval
- Dense features: Semantic segmentation, Monocular depth estimation, Pixel correspondence



The same frozen backbone supports both image-level and pixel-level tasks.

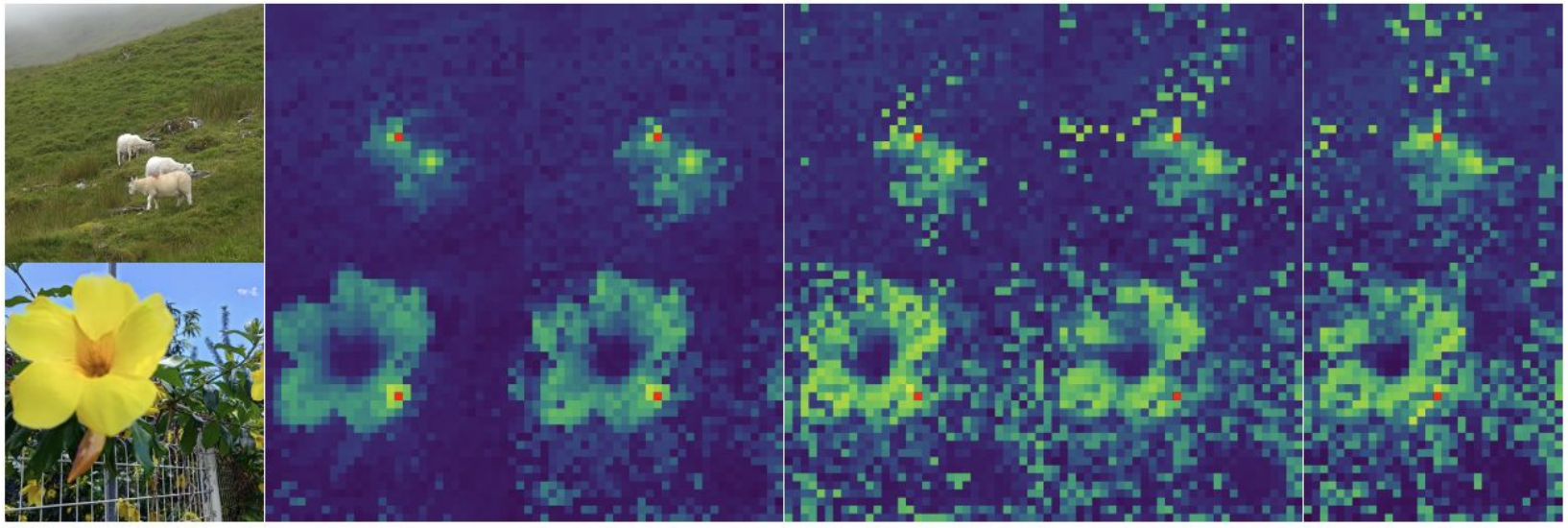
What Does DINOv2 Teach Us?

- The SSL objective alone is not enough.
  - Data quality determines what the model can learn.
  - Patch-level learning improves dense representations.
  - Large models require stable and efficient training.
  - Distillation makes large-scale SSL models practical.

Can we continue scaling DINOv2 indefinitely?

DINOv3: Scaling Reveals a New Problem

- Longer training improves global features
  - Image classification continues to improve.
- Dense features gradually degrade
  - Patch similarity becomes noisy.
  - Segmentation performance decreases.



Gram Anchoring

- Select an earlier model with clean patch features.
- Compute pairwise similarities between its patch features.
- Preserve this similarity structure during longer training.

$$\mathcal{L}_{\text{Gram}} = \left\| \mathbf{X}_S \cdot \mathbf{X}_S^\top - \mathbf{X}_G \cdot \mathbf{X}_G^\top \right\|_{\text{F}}^2 .$$

Gram Anchoring

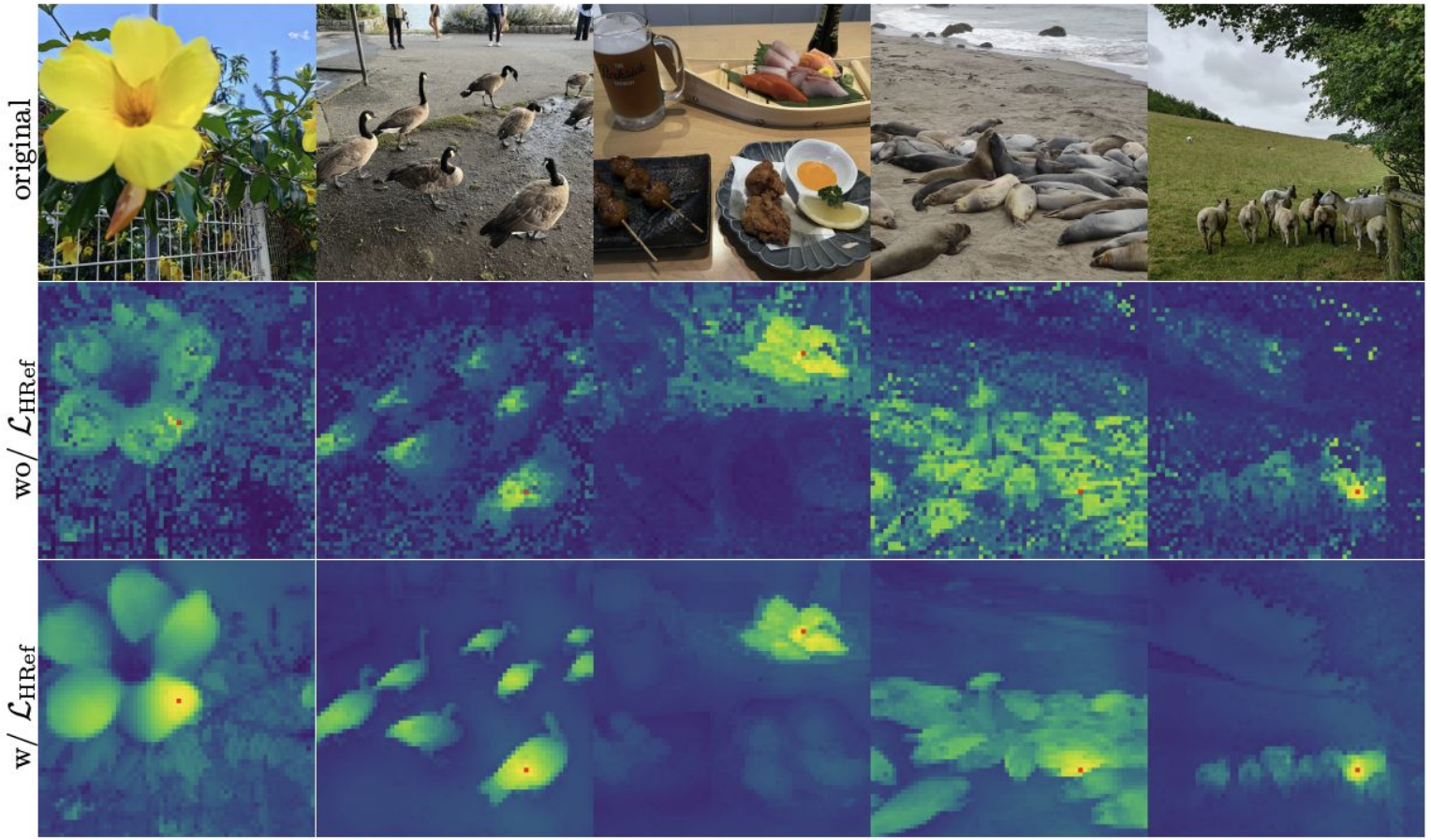


Figure 10: Qualitative effect of Gram anchoring. We visualize cosine maps before and after using the refinement objective $\mathcal{L}_{\text{HRef}}$. The input resolution of the images is 1024×1024 pixels.

Original

- Input image
- Red cross indicates the query patch.

w/o HRef

- Similarity spreads to irrelevant regions.
- Object boundaries become unclear.
- Dense features contain spatial noise.

w/ HRef

- Similarity concentrates on corresponding object parts.
- Background responses decrease.
- Object structure and boundaries become clearer.

Gram anchoring restores clean and spatially consistent patch similarities.

- Can positive views be generated rather than augmented?
- Can SSL scale into a general-purpose visual foundation model?
- Do we still need carefully designed augmentations?

- Recent studies reconsider how views are generated, matched, and scaled.

Next Week..

- Continue on Contrastive Learning ..